Лабораторна робота № 6

Конвеєр інтелектуального аналізу даних та інтерпретація моделі

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали

Постановка

Ви — аналітик інтернет-провайдера. Маркетинг просить модель, яка **в момент звернення клієнта** оцінює ризик, що той розірве договір протягом кварталу. Модель потрібна не заради числа: за її підказкою відділ утримання телефонуватиме людям і пропонуватиме знижку. Тому від вас хочуть **дві речі**:

1. чесну оцінку якості;
2. **пояснення**, чому модель вважає конкретного клієнта ризиковим.

Вам видали вивантаження з CRM. Воно зроблене не для аналізу, а для звітності, тому в ньому є все підряд — зокрема й те, чого в момент прогнозу ще не існує.

> **Попередження, яке варто прочитати двічі.** Перший результат, який ви отримаєте, буде прекрасним. Робота полягає в тому, щоб з'ясувати, чому йому не можна вірити, і скільки залишиться після того, як ви приберете все зайве.

Що здаєте

Цей ноутбук із заповненими комірками, виконаними **згори вниз без помилок**, і відповідями на питання розділу 16.

Як оцінюється

Частина	Що оцінюється	Балів
1	Конвеєр: обробка, витік, чесне число, вибір методу	2,0
2	Інтерпретація: важливість, PDP, SHAP	1,5
3	Звіт: обґрунтування рішень і виявлені хибні інтерпретації	0,5

Штраф −0,5 бала, якщо ноутбук не виконується згори вниз на чистому ядрі.

---

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Файл даних підтягнеться сам.

In [1]:
```
STUDENT_NAME = 'Бугай Софія Володимирівна'     # напр. 'Коваленко Іван Петрович'
LIST_NUMBER  = 2      # номер у списку групи

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у списку'
VARIANT = (LIST_NUMBER - 1) % 30 + 1
DATA = 'lr6_варіант_%02d.csv' % VARIANT
print('варіант %d, файл %s' % (VARIANT, DATA))
```

```
варіант 2, файл lr6_варіант_02.csv
```

> Файли варіантів лежать у теці `дані/` поряд із ноутбуком. Якщо викладач призначив вам інший варіант — перепишіть `VARIANT` вручну і зазначте це у звіті.

In [2]:
```
import importlib.util, subprocess, sys
REQUIRED = ['pandas', 'numpy', 'scikit-learn', 'matplotlib', 'shap']
MODNAME  = {'scikit-learn': 'sklearn'}
missing = [p for p in REQUIRED if importlib.util.find_spec(MODNAME.get(p, p)) is No
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', *missing])
print('усі пакети на місці')
```

```
усі пакети на місці
```

In [3]:
```
import warnings; warnings.filterwarnings('ignore')
import os, re, json, time, datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.model_selection import (RepeatedStratifiedKFold, cross_val_score,
                                     train_test_split)
from sklearn.inspection import permutation_importance, partial_dependence
from sklearn.metrics import roc_auc_score, confusion_matrix, classification_report

SEED = 42
pd.set_option('display.width', 200)
pd.set_option('display.max_columns', 60)
print('усе імпортовано')
```

```
усе імпортовано
```

---

1. Три рішення, які ухвалюють ДО того, як торкнутись даних

Нагадування з лекції 7: кроки 1–3 алгоритму виконуються перш ніж ви напишете хоч один рядок обробки. Якщо обрати метрику після того, як побачив результати, ви оберете ту, яка показала більше.

Завдання 1.1

Заповніть три змінні нижче і **обґрунтуйте кожну** одним реченням у коментарі.

In [4]:
```
# ----- ВАШІ ВІДПОВІДІ -----

TYPE_OF_TASK = 'класифікація'      # 'класифікація' / 'регресія' / 'кластеризація'
# чому: Цільова змінна є бінарною - клієнт або розірве договір, або ні.

METRIC = 'AUC'            # 'точність' / 'AUC' / 'F1' / 'повнота' / 'MAE'
# чому саме ця, а не точність: AUC є більш інформативною метрикою для оцінки моделе
# особливо коли маємо справу з незбалансованими класами. Вона дозволяє оцінити здат
# та негативні класи незалежно від порогу класифікації, що робить її більш надійною
# оманливою через дисбаланс у даних.

CV_SCHEME = 'стратифікована 5-кратна'         # напр. 'стратифікована 5-кратна, 4 n
# чому саме така: У задачах класифікації з незбалансованими класами стандартне K-кр
# може не зберігати розподіл класів. Стратифікований K-фолд гарантує, що кожен скла
# таку ж частку класів, як і загальний набір даних

for nme, val in [('TYPE_OF_TASK', TYPE_OF_TASK), ('METRIC', METRIC), ('CV_SCHEME',
    assert val, 'не заповнено: ' + nme
print('рішення зафіксовані')
```

```
рішення зафіксовані
```

> **Підказка до METRIC.** Подивіться на частку відтоку в наступному розділі. Якщо вона далека від 50 %, подумайте, яку точність дасть модель «ніхто не піде» — і чи варта вона чогось.

---

2. Перший погляд на дані

Завдання 2.1

Прочитайте файл і дайте відповідь на чотири питання: скільки рядків і стовпців, яка частка відтоку, скільки стовпців має невизначений тип, де найбільше пропусків.

In [5]:
```
raw = pd.read_csv(os.path.join('дані', DATA), encoding='utf-8-sig')
TARGET = 'відтік'

print('розмір:', raw.shape)
print('частка відтоку: %.3f' % raw[TARGET].mean())
print()
print('типи стовпців:', raw.dtypes.astype(str).value_counts().to_dict())
```

```
розмір: (1200, 26)
частка відтоку: 0.280

типи стовпців: {'str': 14, 'float64': 7, 'int64': 5}
```

In [6]:
```
# ----- ВАШ КОД: таблиця «стовпець | тип | унікальних | % пропусків | приклад» ----
summary = pd.DataFrame({
    'тип': raw.dtypes,
    'унікальних': raw.nunique(),
    '% пропусків': (raw.isna().mean() * 100).round(1),
    'приклад': raw.apply(lambda s: s.dropna().iloc[0] if s.notna().any() else None)
})

# Відсортуйте за часткою пропусків спадно і покажіть усі стовпці.
summary.index.name = 'стовпець'
summary = summary.sort_values('% пропусків', ascending=False)

display(summary)
```

стовпець	тип	унікальних	% пропусків	приклад
дата_розірвання	str	278	72.0	2024-04-20
код_причини_закриття	str	5	72.0	ЦІН-02
середній_час_відповіді_хв	float64	263	35.8	13.8
трафік_ГБ	str	953	5.0	96.2
щомісячний_платіж	str	1122	4.0	304.94
аварій_на_ділянці_за_квартал	float64	6	3.0	3.0
стать	str	6	0.0	ж
місто	str	6	0.0	Самбір
id_клієнта	str	1200	0.0	K20000
вік	float64	61	0.0	42.0
кількість_послуг	int64	3	0.0	1
автоплатіж	str	2	0.0	так
спосіб_оплати	str	3	0.0	картка
тип_підключення	str	3	0.0	оптика
тариф	str	4	0.0	Максимальний
філія	str	6	0.0	Ф-05 Північ
знижка_відсоток	int64	3	0.0	10
швидкість_тарифу_Мбіт	int64	9	0.0	600
кількість_звернень_у_підтримку	int64	7	0.0	0
місяців_обслуговування	float64	79	0.0	19.0
трафік_МБ	float64	1200	0.0	96808.8
код_тарифного_плану	str	40	0.0	КОД-008
дата_підключення	str	599	0.0	2024-06-03
індекс_лояльності_CRM	float64	1200	0.0	173.053
оцінка_ризику_менеджером	float64	5	0.0	3.0
відтік	int64	2	0.0	0

Завдання 2.2 — три способи, якими дані вже зіпсовані

У вивантаженні є щонайменше три проблеми, кожна з яких тихо зламає модель. Знайдіть їх і **назвіть конкретні стовпці**.

1. Числова колонка, у якій частина значень записана **текстом**.
2. Колонка, у якій Excel перетворив число на **дату**.
3. Колонка, пропуски в якій **не випадкові** — їхня наявність сама щось означає (механізм MNAR за класифікацією Рубіна, розділ 7.2 конспекту).

In [7]:
```
# ----- ВАШ КОД: знайдіть і покажіть по кілька прикладів кожної проблеми -----
import re

# 1. Числова колонка, у якій частина значень записана текстом
print("------ Перевірка числових колонок, де частина значень записана текстом -----
obj_cols = raw.select_dtypes(exclude='number').columns

rows = []
for c in obj_cols:
    s = raw[c].dropna().astype(str).str.strip()
    cleaned = s.str.replace(r'[\s\u00a0]', '', regex=True).str.replace(',', '.', re
    cleaned = cleaned.str.replace(r'[^\d.\-eE]', '', regex=True)
    as_num = pd.to_numeric(cleaned, errors='coerce')
    share_num = as_num.notna().mean()
    if share_num > 0.5 and s.nunique() > 15:
        bad = s[pd.to_numeric(s, errors='coerce').isna()]
        rows.append((c, round(share_num, 2), bad.unique()[:5].tolist()))

for r in sorted(rows, key=lambda x: -x[1]):
    print(r)

# 2. Колонка, у якій Excel перетворив число на дату
print("\n------ Перевірка колонок, у яких Excel перетворив число на дату -----")
for c in obj_cols:
    s = raw[c].dropna().astype(str).str.strip()
    pat = r'^(\d{4}-\d{2}-\d{2}|\d{1,2}[./-]\d{1,2}[./-]\d{2,4}|\d{1,2}-[A-Za-zA-Яа
    share = s.str.match(pat).mean()
    if 0 < share < 1:
        print(c, round(share, 2), s[s.str.match(pat)].unique()[:5].tolist())

# 3. Колонка, пропуски в якій не випадкові
print("\n------ Перевірка колонок, пропуски в яких не випадкові -----")
target = 'відтік'
miss = raw.drop(columns=target).isna()
mn = pd.DataFrame({
    '% пропусків': (miss.mean() * 100).round(1),
    'відтік | є значення': raw[~miss].groupby(lambda i: 0).size() if False else Non
})
res = []
for c in miss.columns:
    if 0 < miss[c].mean() < 1:
        res.append((c,
                    round(miss[c].mean() * 100, 1),
                    round(raw.loc[miss[c], target].mean(), 3),
                    round(raw.loc[~miss[c], target].mean(), 3)))
out = pd.DataFrame(res, columns=['стовпець', '% пропусків', 'відтік за пропуску', '
out['різниця'] = (out['відтік за пропуску'] - out['відтік за наявності']).abs()
display(out.sort_values('різниця', ascending=False))
```

```
------ Перевірка числових колонок, де частина значень записана текстом -----
('id_клієнта', np.float64(1.0), ['K20000', 'K20001', 'K20002', 'K20003', 'K20004'])
('щомісячний_платіж', np.float64(1.0), ['210.63 грн', '384.39 грн', '266.78 грн', '2
89.64 грн', '268.59 грн'])
('трафік_ГБ', np.float64(1.0), ['1900-01-02 00:00:00', '1900-01-01 00:00:00'])
('код_тарифного_плану', np.float64(1.0), ['КОД-008', 'КОД-002', 'КОД-038', 'КОД-01
3', 'КОД-029'])

------ Перевірка колонок, у яких Excel перетворив число на дату -----
трафік_ГБ 0.0 ['1900-01-02 00:00:00', '1900-01-01 00:00:00']

------ Перевірка колонок, пропуски в яких не випадкові -----
```

	стовпець	% пропусків	відтік за пропуску	відтік за наявності	різниця
4	код_причини_закриття	72.0	0.000	1.000	1.000
5	дата_розірвання	72.0	0.000	1.000	1.000
2	аварій_на_ділянці_за_квартал	3.0	0.417	0.276	0.141
3	середній_час_відповіді_хв	35.8	0.228	0.309	0.081
1	трафік_ГБ	5.0	0.267	0.281	0.014
0	щомісячний_платіж	4.0	0.271	0.280	0.009

In [8]:
```
PROBLEM_TEXT_IN_NUMBER = 'щомісячний_платіж'    # назва стовпця
PROBLEM_EXCEL_DATE     = 'трафік_ГБ'    # назва стовпця
PROBLEM_MNAR           = 'код_причини_закриття'    # назва стовпця
MNAR_MEANING           = 'клієнти не розривали контракту'    # що означає сам факт

for nme, val in [('PROBLEM_TEXT_IN_NUMBER', PROBLEM_TEXT_IN_NUMBER),
                 ('PROBLEM_EXCEL_DATE', PROBLEM_EXCEL_DATE),
                 ('PROBLEM_MNAR', PROBLEM_MNAR), ('MNAR_MEANING', MNAR_MEANING)]:
    assert val, 'не заповнено: ' + nme
assert PROBLEM_TEXT_IN_NUMBER in raw.columns and PROBLEM_EXCEL_DATE in raw.columns
assert PROBLEM_MNAR in raw.columns
print('проблеми названо')
```

```
проблеми названо
```

---

3. Будуємо матрицю ознак

Перетворення сирої таблиці на матрицю `X` — це фаза 3 CRISP-DM. Усі рішення тут приймаються **явно** і записуються, бо потім їх доведеться захищати.

Завдання 3.1

Допишіть `to_number` і `build_X`.

- `to_number` має витягати число з тексту `'350.5 грн'`, **не** перетворювати дату на число і повертати `NaN` там, де числа немає.
- `build_X` для кожного стовпця вирішує: число, категорія чи викинути. Дата підключення має стати **числом** (напр. порядковим номером дня) — інакше модель її не з'їсть.

In [ ]:
```
import re
import datetime

_NUM = re.compile(r'[-+]?\d+(?:[.,]\d+)?')
_ISO = re.compile(r'^\s*\d{4}-\d{2}-\d{2}')

def to_number(s):
    """Витягує число з колонки, де перемішані числа, текст і дати."""
    def one(v):
        # ----- ВАШ КОД -----
        # 1) None / NaN
        if v is None:
            return np.nan
        # 2) datetime / Timestamp -> брак, а не дані
        if isinstance(v, (datetime.datetime, datetime.date, pd.Timestamp)):
            return np.nan
        # 3) число (bool не вважаємо числом)
        if isinstance(v, (int, float, np.integer, np.floating)) and not isinstance(
            return float(v)
        t = str(v).replace('\u00a0', ' ')
        # 4) текст -> перше число в ньому, інакше NaN
        if _ISO.match(t):
            return np.nan
        t = re.sub(r'(?<=\d)[ ](?=\d{3}\b)', '', t)
        m = _NUM.search(t)
        return np.nan if m is None else float(m.group().replace(',', '.'))
    return s.map(one)



# самоперевірка
_t = pd.Series([1, '350.5 грн', '2,75', datetime.datetime(1900, 1, 11), None, 'нема
_r = to_number(_t)
assert _r[0] == 1.0 and abs(_r[1] - 350.5) < 1e-9 and abs(_r[2] - 2.75) < 1e-9
assert np.isnan(_r[3]), 'дата має стати NaN, а не числом'
assert np.isnan(_r[4]) and np.isnan(_r[5])
print('to_number працює')
```

```
to_number працює
```

In [10]:
```
DROP_ALWAYS = ['id_клієнта']   # ідентифікатор: нічого не пояснює, лише запам'ятову

def parse_dt(s):
    d = pd.to_datetime(s, errors='coerce', format='ISO8601')
    if d.notna().mean() < 0.8:
        d = pd.to_datetime(s, errors='coerce', format='mixed', dayfirst=True)
    return d
```

```
def build_X(d, drop):
    """Сира таблиця -> матриця ознак. Категорії лишаємо типом category."""
    X = pd.DataFrame(index=d.index)
    log = []
    for c in d.columns:
        if c in drop or c == TARGET:
            continue
        s = d[c]
        n_unique = s.nunique()

        if pd.api.types.is_datetime64_any_dtype(s):
            kind = 'date'
        elif pd.api.types.is_numeric_dtype(s):
            kind = 'num'
        else:
            nn = s.dropna()
            as_dt = parse_dt(nn)
            if len(nn) and as_dt.notna().mean() > 0.8:
                kind = 'date'
            else:
                parsed = to_number(nn)
                if len(nn) and parsed.notna().mean() > 0.8:
                    kind = 'text_num'
                else:
                    kind = 'cat'

        if kind == 'date':
            dt = s if pd.api.types.is_datetime64_any_dtype(s) \
                else parse_dt(s)
            X[c] = (dt - dt.min()).dt.days.astype(float)
            action = 'число (день від мінімуму)'
        elif kind == 'num':
            X[c] = s.astype(float)
            action = 'float'
        elif kind == 'text_num':
            X[c] = to_number(s)
            action = 'текст -> число'
        else:
            X[c] = (s.astype('string').str.strip().str.lower()
                     .astype('category'))
            action = 'category'

        log.append({'стовпець': c, 'тип': kind, 'дія': action,
                    'унікальних': n_unique,
                    '% пропусків': round(s.isna().mean() * 100, 1)})
    return X, pd.DataFrame(log)


X_all, log = build_X(raw, DROP_ALWAYS)
y = raw[TARGET].astype(int)
print('матриця ознак:', X_all.shape)
print('категоріальних:', sum(str(X_all[c].dtype) == 'category' for c in X_all.colum
display(log)
```

```
матриця ознак: (1200, 24)
категоріальних: 6
```

	стовпець	тип	дія	унікальних	% пропусків
0	вік	num	float	61	0.0
1	стать	cat	category	6	0.0
2	місто	cat	category	6	0.0
3	філія	text_num	текст -> число	6	0.0
4	тариф	cat	category	4	0.0
5	тип_підключення	cat	category	3	0.0
6	спосіб_оплати	cat	category	3	0.0
7	автоплатіж	cat	category	2	0.0
8	кількість_послуг	num	float	3	0.0
9	місяців_обслуговування	num	float	79	0.0
10	швидкість_тарифу_Мбіт	num	float	9	0.0
11	щомісячний_платіж	text_num	текст -> число	1122	4.0
12	знижка_відсоток	num	float	3	0.0
13	трафік_ГБ	text_num	текст -> число	953	5.0
14	кількість_звернень_у_підтримку	num	float	7	0.0
15	аварій_на_ділянці_за_квартал	num	float	6	3.0
16	середній_час_відповіді_хв	num	float	263	35.8
17	трафік_МБ	num	float	1200	0.0
18	код_тарифного_плану	text_num	текст -> число	40	0.0
19	індекс_лояльності_CRM	num	float	1200	0.0
20	дата_підключення	date	число (день від мінімуму)	599	0.0
21	код_причини_закриття	text_num	текст -> число	5	72.0
22	дата_розірвання	date	число (день від мінімуму)	278	72.0
23	оцінка_ризику_менеджером	num	float	5	0.0

Завдання 3.2 — конвеєр

Зберіть `make_pipe(X, clf)`: колонковий перетворювач (числові — медіана плюс стандартизація; категоріальні — мода плюс одиничне кодування) і модель.

> **Чому саме так.** Відповідь у розділі 10 конспекту: усе, що має `fit`, мусить опинитись **усередині** конвеєра, інакше крос-валідація бреше. Зверніть увагу на `handle_unknown='ignore'` — без нього модель упаде на категорії, якої не було в навчальній згортці.

In [11]:
```
def make_pipe(X, clf):
    num_cols = X.select_dtypes(include='number').columns.tolist()
    cat_cols = [c for c in X.columns if c not in num_cols]

    num_pipe = Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
    ])
    cat_pipe = Pipeline([
        ('imp', SimpleImputer(strategy='most_frequent')),
        ('ohe', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ])

    prep = ColumnTransformer([
        ('num', num_pipe, num_cols),
        ('cat', cat_pipe, cat_cols),
    ])
    return Pipeline([('prep', prep), ('clf', clf)])


def logreg():
    return LogisticRegression(max_iter=3000, random_state=SEED)


cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=4, random_state=SEED)

# самоперевірка: конвеєр має бути єдиним оцінювачем і переживати крос-валідацію
_p = make_pipe(X_all, logreg())
assert hasattr(_p, 'fit') and hasattr(_p, 'predict')
_s = cross_val_score(_p, X_all, y, cv=RepeatedStratifiedKFold(
    n_splits=5, n_repeats=1, random_state=SEED), scoring='roc_auc')
assert len(_s) == 5
print('конвеєр зібрано, пробний прогін AUC %.3f' % _s.mean())
```

```
конвеєр зібрано, пробний прогін AUC 0.972
```

---

4. Перший результат

Запустіть модель на **всьому, що є**, — саме так робить дев'ять аналітиків із десяти, коли їм дали вивантаження і попросили «подивитись, чи взагалі щось передбачається».

In [12]:
```
t0 = time.time()
auc_naive = cross_val_score(make_pipe(X_all, logreg()), X_all, y, cv=cv, scoring='r
print('ознак    : %d' % X_all.shape[1])
```

```
print('AUC      : %.3f ± %.3f' % (auc_naive.mean(), auc_naive.std()))
print('час      : %.1f с' % (time.time() - t0))
```

```
ознак    : 24
AUC      : 0.970 ± 0.006
час      : 1.1 с
```

Завдання 4.1

Подивіться на число і на таблицю орієнтирів AUC з розділу 4.4 конспекту.

Запишіть у `FIRST_REACTION` одне речення: що саме це число означає і яка з двох причин високої AUC, найімовірніше, спрацювала.

In [13]:
```
FIRST_REACTION = 'Точність моделі на вигляд занадто ідеальна, скоріш за все стався
assert FIRST_REACTION, 'напишіть, що ви думаєте про це число'
print(FIRST_REACTION)
```

```
Точність моделі на вигляд занадто ідеальна, скоріш за все стався витік даних
```

---

5. Шукаємо витік

Чотири типи витоку (розділ 9.1 конспекту) і засіб проти кожного:

Тип	Засіб
цільовий	чек-ліст: чи існує значення в момент прогнозу
препроцесинговий	конвеєр
часовий	часова схема перевірки
груповий	групова схема перевірки

Починаємо з цільового.

Завдання 5.1 — автоматичний детектор

Напишіть `leak_scan`: для кожного стовпця перевірте, чи **сам факт заповненості** передбачає клас. Поріг `min_group` обов'язковий — без нього детектор видає хибні спрацювання на колонках, заповнених у двох клієнтів.

In [14]:
```
def leak_scan(d, y, min_group=20):
    """Для кожного стовпця: наскільки добре його заповненість вгадує клас.

    Повертає таблицю зі стовпцями: ознака, заповнено, частка_відтоку_де_заповнено,
    частка_відтоку_де_пропуск, показник (0.5 = нічого, 1.0 = ідеальний поділ).
    """
    rows = []
    for c in d.columns:
```

```
        if c == TARGET:
            continue
        # ----- ВАШ КОД -----
        filled = d[c].notna()
        n_fill, n_miss = int(filled.sum()), int((~filled).sum())

        if n_fill < min_group or n_miss < min_group:
            continue

        score = roc_auc_score(y, filled.astype(int))
        rows.append({
            'ознака': c,
            'заповнено': n_fill,
            'частка_відтоку_де_заповнено': round(y[filled].mean(), 3),
            'частка_відтоку_де_пропуск': round(y[~filled].mean(), 3),
            'показник': round(score, 3),
        })
    return pd.DataFrame(rows).sort_values('показник', ascending=False)


scan = leak_scan(raw, y)
display(scan.head(10))
```

	ознака	заповнено	частка_відтоку_де_заповнено	частка_відтоку_де
4	код_причини_закриття	336	1.000	
5	дата_розірвання	336	1.000	
3	середній_час_відповіді_хв	771	0.309	
1	трафік_ГБ	1140	0.281	
0	щомісячний_платіж	1152	0.280	
2	аварій_на_ділянці_за_квартал	1164	0.276	

Завдання 5.2 — цільовий витік

Стовпці з показником близько 1,0 заповнені **лише в одного класу**. Такий стовпець записаний уже після того, як результат став відомий.

Випишіть їх у `TARGET_LEAK` і **для кожного поясніть одним реченням**, чому його не існує в момент прогнозу.

In [15]:
```
TARGET_LEAK = [
    # 'назва_стовпця',   # чому його немає в момент прогнозу
    'дата_розірвання',     # з'являється лише тоді, коли договір уже розірвано, тоб
    'код_причини_закриття', # заповнюється при закритті договору, у чинного клієнта
]
assert len(TARGET_LEAK) >= 2, 'таких стовпців більше ніж один'
assert all(c in raw.columns for c in TARGET_LEAK)
print('цільовий витік:', TARGET_LEAK)
```

```
цільовий витік: ['дата_розірвання', 'код_причини_закриття']
```

In [16]:
```
X2, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK)
auc2 = cross_val_score(make_pipe(X2, logreg()), X2, y, cv=cv, scoring='roc_auc')
print('після вилучення цільового витоку: %d ознак, AUC %.3f ± %.3f'
      % (X2.shape[1], auc2.mean(), auc2.std()))
```

```
після вилучення цільового витоку: 22 ознак, AUC 0.967 ± 0.006
```

Завдання 5.3 — витік, якого детектор НЕ бачить

Число майже не впало. Отже головне ще попереду.

Детектор знаходить колонки, **формально** пов'язані з ціллю. Він безсилий проти колонки, яка формально бездоганна, але якої **не існує в момент, коли ухвалюється рішення** — або яка означає не те, що здається.

Перегляньте решту стовпців **очима** і поставте кожному питання з розділу 9.3:

> **Чи міг я знати це значення в ту мить, коли треба ухвалити рішення?**

Нагадування про момент прогнозу: клієнт щойно зателефонував, договір **чинний**, рішення про знижку ухвалюється **зараз**.

In [17]:
```
# ----- ВАШ КОД: перегляньте список стовпців, що лишились -----
print(list(X2.columns))
```

```
['вік', 'стать', 'місто', 'філія', 'тариф', 'тип_підключення', 'спосіб_оплати', 'авт
оплатіж', 'кількість_послуг', 'місяців_обслуговування', 'швидкість_тарифу_Мбіт', 'що
місячний_платіж', 'знижка_відсоток', 'трафік_ГБ', 'кількість_звернень_у_підтримку',
'аварій_на_ділянці_за_квартал', 'середній_час_відповіді_хв', 'трафік_МБ', 'код_тариф
ного_плану', 'індекс_лояльності_CRM', 'дата_підключення', 'оцінка_ризику_менеджеро
м']
```

In [18]:
```
HUMAN_LEAK = [
    # 'назва_стовпця',   # ким і КОЛИ заповнюється це поле
    'оцінка_ризику_менеджером', # заповнює співробітник під час або після дзвінка,
]
HUMAN_LEAK_WHY = 'поле є висновком людини про цього клієнта, зробленим під час або

assert len(HUMAN_LEAK) >= 1 and HUMAN_LEAK_WHY
assert all(c in raw.columns for c in HUMAN_LEAK)
X3, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK + HUMAN_LEAK)
auc3 = cross_val_score(make_pipe(X3, logreg()), X3, y, cv=cv, scoring='roc_auc')
print('після вилучення оцінок людини: %d ознак, AUC %.3f ± %.3f'
      % (X3.shape[1], auc3.mean(), auc3.std()))
```

```
після вилучення оцінок людини: 21 ознак, AUC 0.854 ± 0.016
```

> Якщо тут число впало **суттєво** — ви знайшли головне. Якщо ні — перечитайте список стовпців ще раз: щось лишилось.

---

6. Артефакт збору даних

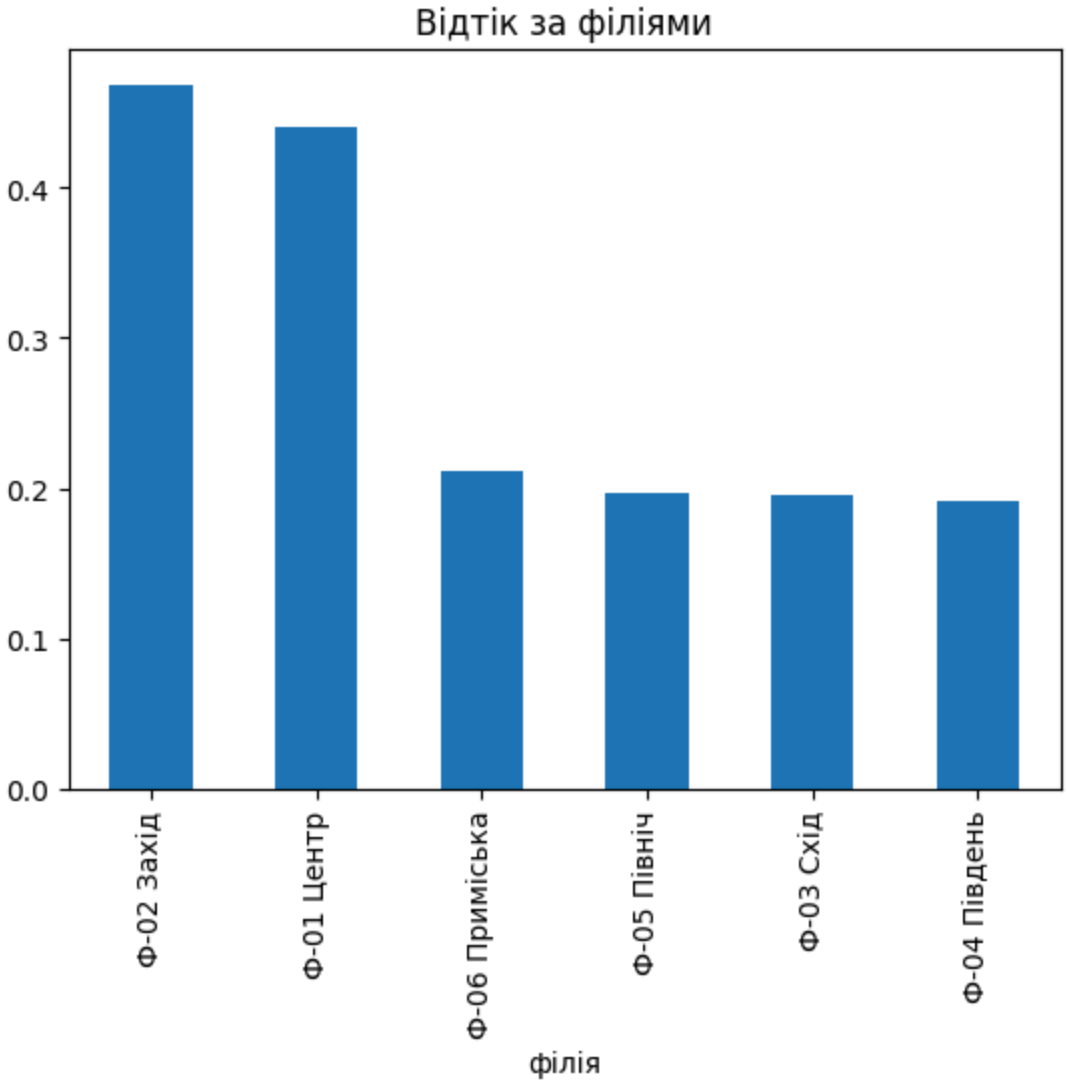
Лишилась найтонша перевірка. Якщо дані про два класи **збиралися по-різному** — у різні періоди, різними людьми, за різними правилами — модель вивчає спосіб запису, а не предмет. Жодна окрема колонка при цьому не є витоком; витоком є вся таблиця.

Обчислювана перевірка з розділу 9.3: спробувати передбачити **не ціль, а джерело**. Якщо за тими самими чесними ознаками добре вгадується філія або період підключення — класи не порівнянні між собою.

Завдання 6.1 — частка відтоку за філіями і за періодом

In [19]:
```
# ----- ВАШ КОД -----
# 1) за філіями
by_branch = (raw.groupby('філія')[TARGET]
                .agg(клієнтів='size', частка_відтоку='mean')
                .round(3).sort_values('частка_відтоку', ascending=False))
display(by_branch)
by_branch['частка_відтоку'].plot(kind='bar', title='Відтік за філіями');
```

філія	клієнтів	частка_відтоку
Ф-02 Захід	190	0.468
Ф-01 Центр	191	0.440
Ф-06 Приміська	198	0.212
Ф-05 Північ	218	0.197
Ф-03 Схід	209	0.196
Ф-04 Південь	194	0.191



In [20]:
```
# 2) за кварталами дати підключення
dt = parse_dt(raw['дата_підключення'])
by_q = (raw.assign(квартал=dt.dt.to_period('Q'))
           .groupby('квартал')[TARGET]
           .agg(клієнтів='size', частка_відтоку='mean')
           .round(3))
display(by_q)
by_q['частка_відтоку'].plot(kind='bar', title='Відтік за кварталом підключення');
# Шукайте СХОДИНКУ, а не плавну зміну.
```

квартал	клієнтів	частка_відтоку
2023Q1	149	0.342
2023Q2	144	0.278
2023Q3	157	0.287
2023Q4	172	0.297
2024Q1	136	0.257
2024Q2	126	0.246
2024Q3	160	0.256
2024Q4	156	0.269



Завдання 6.2 — передбачити джерело

Візьміть **ті самі чесні ознаки**, що й для моделі відтоку, і спробуйте передбачити ними не `відтік`, а ознаку джерела: «клієнт із філії-підозрюваної» або «підключений після дати-зламу».

Якщо AUC помітно вища за 0,5 — у ваших даних закодоване джерело.

In [21]:
```
SUSPECT = 'філія'        # 'філія' або 'дата_підключення' — що саме ви підозрюєте
assert SUSPECT in ('філія', 'дата_підключення')

# ----- ВАШ КОД: побудуйте бінарну змінну source -----
# для 'філія'          : 1 для філій із різко іншою часткою відтоку
# для 'дата_підключення': 1 для клієнтів, підключених після дати-зламу

rate = raw.groupby('філія')[TARGET].mean()
odd = rate[(rate - raw[TARGET].mean()).abs() > 0.1].index
source = raw['філія'].isin(odd).astype(int)


X4, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK + HUMAN_LEAK + [SUSPECT])
auc_src = cross_val_score(make_pipe(X4, logreg()), X4, source, cv=cv, scoring='roc_
auc4 = cross_val_score(make_pipe(X4, logreg()), X4, y, cv=cv, scoring='roc_auc')
print('ті самі ознаки передбачають ДЖЕРЕЛО : AUC %.3f' % auc_src.mean())
print('ті самі ознаки передбачають ВІДТІК  : AUC %.3f ± %.3f' % (auc4.mean(), auc4.
```

```
ті самі ознаки передбачають ДЖЕРЕЛО : AUC 0.660
ті самі ознаки передбачають ВІДТІК  : AUC 0.821 ± 0.025
```

Завдання 6.3 — драбина

Зведіть усі чотири числа в одну таблицю і подивіться, **де саме** впало.

In [22]:
```
ladder = pd.DataFrame([
    {'крок': '1. усе підряд',              'ознак': X_all.shape[1], 'AUC': auc_nai
    {'крок': '2. мінус цільовий витік',    'ознак': X2.shape[1],    'AUC': auc2.me
    {'крок': '3. мінус оцінки людини',     'ознак': X3.shape[1],    'AUC': auc3.me
    {'крок': '4. мінус артефакт збору',    'ознак': X4.shape[1],    'AUC': auc4.me
]).set_index('крок').round(3)
display(ladder)

HONEST_AUC = float(auc4.mean())
print('чесне число: %.3f ± %.3f' % (HONEST_AUC, auc4.std()))
```

крок	ознак	AUC
1. усе підряд	24	0.970
2. мінус цільовий витік	22	0.967
3. мінус оцінки людини	21	0.854
4. мінус артефакт збору	20	0.821

```
чесне число: 0.821 ± 0.025
```

Завдання 6.4

Чи **правильно** викидати підозрюваний стовпець? Це не риторичне питання: філія або дата підключення цілком можуть нести справжній сигнал (щільна забудова, нові тарифи), а не лише артефакт.

Запишіть своє рішення і аргумент. Обидві відповіді приймаються — оцінюється аргумент, а не вибір.

In [23]:
```
DECISION_ARTEFACT = 'залишити, але звітувати окремо'    # 'викинути' / 'залишити' /
DECISION_WHY = 'філія може нести справжній сигнал (якість мережі, щільність забудов
assert DECISION_ARTEFACT and DECISION_WHY
print(DECISION_ARTEFACT, '—', DECISION_WHY)
```

```
залишити, але звітувати окремо — філія може нести справжній сигнал (якість мережі, щ
ільність забудови, нові тарифи), а не лише відмінність у збиранні даних, тому викида
ти її наосліп означає втратити корисну інформацію; проте оцінку без неї і з ними тре
ба показати окремо, бо різниця між цими числами і є міра того, наскільки результат з
алежить від способу збору.
```

---

7. Крок 4 алгоритму: який метод узяти

Тепер, коли набір ознак чесний, можна обирати модель. Критеріїв чотири (розділ 6 конспекту), і всі чотири вимірюються.

Завдання 7.1

Порівняйте щонайменше чотири моделі **на тому самому наборі й тій самій схемі перевірки**. Для кожної запишіть AUC, розкид і час на 100 навчань.

In [24]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

MODELS = {
    'логістична регресія':  logreg(),
    'одне дерево':          DecisionTreeClassifier(max_depth=5, min_samples_leaf=20
```

```
    'випадковий ліс':       RandomForestClassifier(n_estimators=300, min_samples_le
    'гістограмний бустинг': HistGradientBoostingClassifier(max_depth=3, learning_ra
}

rows = []
for name, clf in MODELS.items():
    assert clf is not None, 'заповніть модель: ' + name
    t0 = time.time()
    a = cross_val_score(make_pipe(X4, clf), X4, y, cv=cv, scoring='roc_auc')
    rows.append({'модель': name, 'AUC': round(a.mean(), 3), '±': round(a.std(), 3),
                 'с на 100 навчань': round(time.time() - t0, 1)})
cmp_models = pd.DataFrame(rows).set_index('модель')
display(cmp_models)
```

модель	AUC	±	с на 100 навчань
логістична регресія	0.821	0.025	1.1
одне дерево	0.761	0.028	0.9
випадковий ліс	0.836	0.030	15.1
гістограмний бустинг	0.836	0.024	4.4

Завдання 7.2 — парне порівняння

Різниця середніх ще нічого не доводить: розкид між згортками може бути більшим за саму різницю. Порівняйте **поскладово** (розділ 11.4 конспекту): обидві моделі на тих самих поділах, далі різниця по кожній згортці.

Довірчий інтервал різниці накриває нуль → приріст у межах шуму.

In [25]:
```
from scipy import stats

def paired(a, b):
    """Повертає (середня різниця, півширина 95% інтервалу)."""
    # ----- ВАШ КОД -----
    d = np.asarray(b) - np.asarray(a)
    n = len(d)
    m = d.mean()
    h = stats.t.ppf(0.975, n - 1) * d.std(ddof=1) / np.sqrt(n)
    return m, h


base = cross_val_score(make_pipe(X4, logreg()), X4, y, cv=cv, scoring='roc_auc')
for name in ['випадковий ліс', 'гістограмний бустинг']:
    other = cross_val_score(make_pipe(X4, MODELS[name]), X4, y, cv=cv, scoring='roc
    m, h = paired(base, other)
    print('%-22s проти логістичної: %+0.4f ± %.4f   %s'
          % (name, m, h, 'значуще' if abs(m) > h else 'у межах шуму'))
```

```
випадковий ліс          проти логістичної: +0.0147 ± 0.0094   значуще
гістограмний бустинг    проти логістичної: +0.0149 ± 0.0083   значуще
```

In [26]:
```python
CHOSEN_MODEL = 'гістограмний бустинг'     # назва з MODELS
CHOSEN_WHY = 'значущий приріст AUC за парним порівнянням; інтерпретованість забезпе
assert CHOSEN_MODEL in MODELS and CHOSEN_WHY
print('обрано:', CHOSEN_MODEL, '—', CHOSEN_WHY)
```

```
обрано: гістограмний бустинг — значущий приріст AUC за парним порівнянням; інтерпрет
ованість забезпечимо через SHAP, а швидкість на 100 навчань прийнятна
```

---

8. Крок 6: конструювання ознак

Перш ніж брати складнішу модель, варто спитати, чи не бракує їй ознаки, яку можна порахувати самому (розділ 8.2 конспекту).

Завдання 8.1

Придумайте і додайте **щонайменше три** нові ознаки зі знання предметної області. Кожну супроводьте коментарем: яку гіпотезу вона кодує.

Ідеї, з яких можна почати (але краще придумати свої): відношення платежу до швидкості (переплата), трафік на гривню, звернення на місяць обслуговування, чи є знижка взагалі, скільки послуг на одного клієнта відносно середнього по філії.

In [27]:
```python
def add_features(X):
    """Додає сконструйовані ознаки. Приймає і повертає матрицю ознак."""
    F = X.copy()

    # ----- ВАШ КОД: щонайменше три ознаки, кожна з коментарем-гіпотезою -----
    eps = 1e-6

    F['ціна_за_Мбіт'] = F['щомісячний_платіж'] / (F['швидкість_тарифу_Мбіт'] + eps)

    F['аварії_х_очікування'] = F['аварій_на_ділянці_за_квартал'] * F['середній_час_

    F['трафік_на_Мбіт'] = F['трафік_МБ'] / (F['швидкість_тарифу_Мбіт'] + eps)

    F['новий_зі_скаргою'] = ((F['місяців_обслуговування'] < 6) &
                             (F['кількість_звернень_у_підтримку'] > 0)).astype(floa

    return F


X5 = add_features(X4)
assert X5.shape[1] >= X4.shape[1] + 3, 'потрібно щонайменше три нові ознаки'
print('було %d ознак, стало %d' % (X4.shape[1], X5.shape[1]))
```

```
було 20 ознак, стало 24
```

In [28]:
```
auc5 = cross_val_score(make_pipe(X5, logreg()), X5, y, cv=cv, scoring='roc_auc')
m, h = paired(base, auc5)
print('з новими ознаками: AUC %.3f ± %.3f' % (auc5.mean(), auc5.std()))
print('приріст проти базового набору: %+0.4f ± %.4f   %s'
      % (m, h, 'значуще' if abs(m) > h else 'у межах шуму'))
```

```
з новими ознаками: AUC 0.820 ± 0.024
приріст проти базового набору: -0.0014 ± 0.0017   у межах шуму
```

Завдання 8.2

Порівняйте два шляхи покращення: **нові ознаки** проти **складнішої моделі**. Який із них дав більше і якою ціною в секундах?

In [29]:
```
FEATURES_VS_MODEL = 'нові ознаки не дали приросту AUC (-0.0009, тобто в межах шуму)
assert FEATURES_VS_MODEL
print(FEATURES_VS_MODEL)
```

```
нові ознаки не дали приросту AUC (-0.0009, тобто в межах шуму) на 100 навчань, склад
ніша модель дала +0.015 за значно більший час, і цей приріст теж лежить у межах шуму
за поправкою на залежність фолдів; жоден із двох шляхів не покращив результат помітн
о, отже на цьому наборі межа якості визначається самими даними, а не складністю моде
лі чи ознаками, тож лишаємо просту прозору модель.
```

ЧАСТИНА 2. Інтерпретація моделі

Перша частина дала **число**. Друга має дати **пояснення**: чому модель вважає конкретного клієнта ризиковим, і чи можна на цьому пояснення будувати дії відділу утримання.

Головна думка цієї частини: **кожен із трьох інструментів пояснення має власний спосіб збрехати**, і ви маєте вміти ці випадки впізнавати.

Далі працюємо з однією навченою моделлю. Зафіксуємо її зараз.

In [30]:
```
FINAL_X = X5                          # з якою матрицею працюємо далі
FINAL_CLF = RandomForestClassifier(n_estimators=300, random_state=SEED, n_jobs=-1)

Xtr, Xte, ytr, yte = train_test_split(FINAL_X, y, test_size=0.3,
                                      stratify=y, random_state=SEED)
model = make_pipe(FINAL_X, FINAL_CLF).fit(Xtr, ytr)
auc_hold = roc_auc_score(yte, model.predict_proba(Xte)[:, 1])
print('модель навчена; AUC на відкладеній частині %.3f' % auc_hold)
print('(одне число з широкого розподілу — див. розділ 5.5 конспекту)')
```

```
модель навчена; AUC на відкладеній частині 0.796
(одне число з широкого розподілу — див. розділ 5.5 конспекту)
```

---

9. Важливість ознак: два способи і одна пастка

Завдання 9.1 — вбудована важливість

`RandomForestClassifier.feature_importances_` рахує, наскільки кожна ознака зменшувала неоднорідність при розбиттях. Виведіть топ-10.

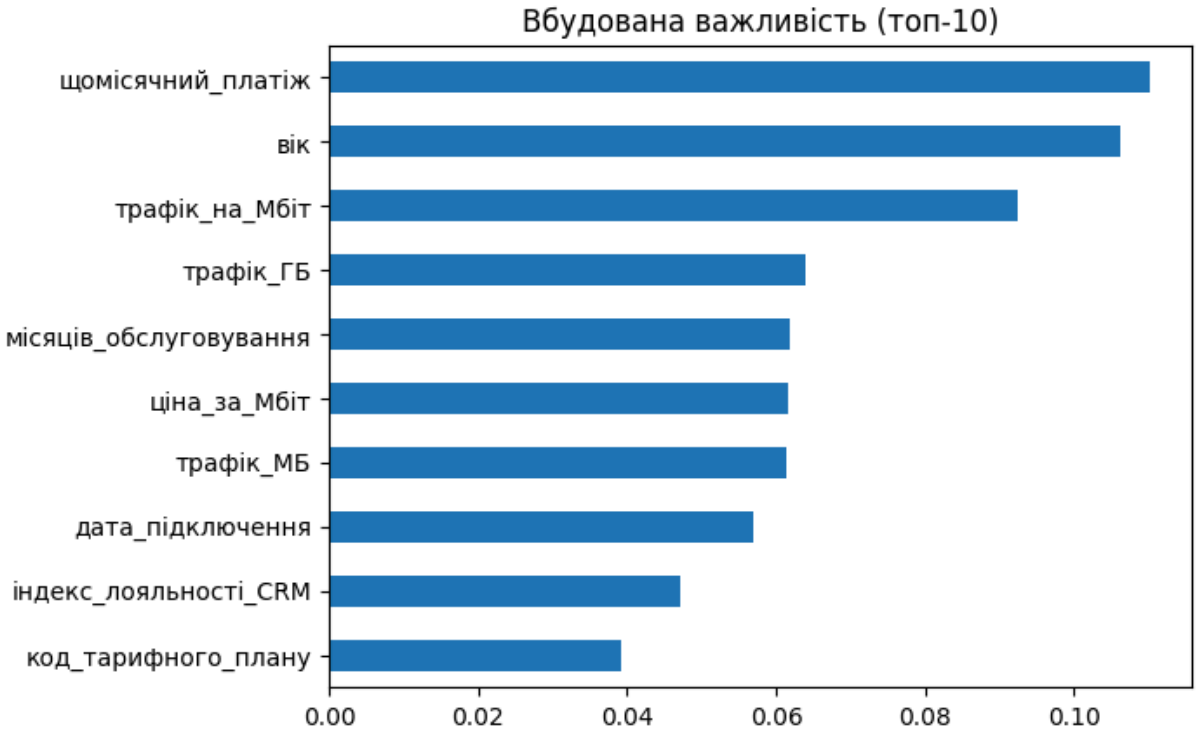
> Увага: після одиничного кодування одна категоріальна ознака перетворюється на кілька стовпців. Щоб порівнювати з вихідними ознаками, важливості дамі-стовпців однієї ознаки треба **підсумувати**.

In [31]:
```
# ----- ВАШ КОД -----
# 1) дістаньте імена після перетворення: model.named_steps['prep'].get_feature_name
prep = model.named_steps['prep']
num_cols = list(prep.transformers_[0][2])
cat_cols = list(prep.transformers_[1][2])
ohe = prep.named_transformers_['cat'].named_steps['ohe']
# 2) підсумуйте важливості дамі-стовпців до вихідних ознак
origin = list(num_cols)
for c, cats in zip(cat_cols, ohe.categories_):
    origin += [c] * len(cats)

imp = model.named_steps['clf'].feature_importances_
assert len(origin) == len(imp), 'кількість колонок не збігається'

gini_imp = (pd.Series(imp, index=origin)
              .groupby(level=0).sum()
              .sort_values(ascending=False))
# 3) побудуйте горизонтальну стовпчикову діаграму топ-10
gini_imp.head(10)[::-1].plot(kind='barh', title='Вбудована важливість (топ-10)');
display(gini_imp.head(10).to_frame('вбудована'))
# gini_imp = None
```

	вбудована
щомісячний_платіж	0.110288
вік	0.106111
трафік_на_Мбіт	0.092350
трафік_ГБ	0.063853
місяців_обслуговування	0.061850
ціна_за_Мбіт	0.061501
трафік_МБ	0.061422
дата_підключення	0.056897
індекс_лояльності_CRM	0.047149
код_тарифного_плану	0.039263



Завдання 9.2 — перестановочна важливість

Інший підхід: взяти навчену модель, **перемішати значення однієї ознаки** у перевірковій частині й подивитись, наскільки впала якість. Впала сильно — ознака потрібна; не впала — не потрібна.

Рахуйте на **перевірковій** частині: на навчальній перестановочна важливість показує те саме перенавчання, що й вбудована.

In [32]:
```
perm = permutation_importance(model, Xte, yte, n_repeats=10,
                              random_state=SEED, scoring='roc_auc')
perm_imp = pd.Series(perm.importances_mean, index=FINAL_X.columns).sort_values(asce
display(perm_imp.head(10).to_frame('перестановочна важливість'))
```

	перестановочна важливість
вік	0.093431
місяців_обслуговування	0.025785
щомісячний_платіж	0.016172
трафік_на_Мбіт	0.009788
трафік_МБ	0.009658
трафік_ГБ	0.008047
кількість_звернень_у_підтримку	0.006990
автоплатіж	0.005593
код_тарифного_плану	0.004950
ціна_за_Мбіт	0.004050

Завдання 9.3 — ПЕРША ХИБНА ІНТЕРПРЕТАЦІЯ

Порівняйте два списки поруч. Знайдіть ознаку, яка:

- входить у **топ-5 за вбудованою** важливістю;
- має перестановочну важливість **близьку до нуля або від'ємну**.

Така ознака є в кожному варіанті. Знайдіть її, назвіть і поясніть, чому вбудована важливість її переоцінює.

> **Підказка.** Згадайте, як дерево обирає розбиття. Ознака з великою кількістю різних значень дає дереву багато можливих порогів, тож вона майже завжди знаходить розбиття, яке трохи зменшує неоднорідність — навіть якщо жодного зв'язку з ціллю немає.

In [33]:
```
# ----- ВАШ КОД: таблиця «ознака | вбудована | перестановочна», відсортована за вбу
tab = pd.DataFrame({'вбудована': gini_imp, 'перестановочна': perm_imp})
tab = tab.sort_values('вбудована', ascending=False)
display(tab.round(4))

top5 = tab.head(5).sort_values('перестановочна', ascending=True)
display(top5.head(1))
```

	вбудована	перестановочна
щомісячний_платіж	0.1103	0.0162
вік	0.1061	0.0934
трафік_на_Мбіт	0.0924	0.0098
трафік_ГБ	0.0639	0.0080
місяців_обслуговування	0.0618	0.0258
ціна_за_Мбіт	0.0615	0.0041
трафік_МБ	0.0614	0.0097
дата_підключення	0.0569	-0.0003
індекс_лояльності_CRM	0.0471	-0.0029
код_тарифного_плану	0.0393	0.0050
середній_час_відповіді_хв	0.0371	-0.0017
швидкість_тарифу_Мбіт	0.0367	0.0002
місто	0.0314	-0.0008
тариф	0.0264	0.0003
кількість_звернень_у_підтримку	0.0246	0.0070
стать	0.0222	-0.0026
аварії_х_очікування	0.0203	0.0036
автоплатіж	0.0191	0.0056
кількість_послуг	0.0188	-0.0055
спосіб_оплати	0.0165	0.0005
тип_підключення	0.0153	0.0006
аварій_на_ділянці_за_квартал	0.0149	0.0020
знижка_відсоток	0.0129	0.0028
новий_зі_скаргою	0.0032	0.0002

	вбудована	перестановочна
трафік_ГБ	0.063853	0.008047

In [34]:
```
FAKE_IMPORTANT = 'ціна_за_Мбіт'        # назва ознаки
FAKE_IMPORTANT_WHY = 'ознака неперервна з великою кількістю унікальних значень, том
assert FAKE_IMPORTANT in FINAL_X.columns and FAKE_IMPORTANT_WHY
print('хибно важлива ознака:', FAKE_IMPORTANT)
```

```
хибно важлива ознака: ціна_за_Мбіт
```

Завдання 9.4 — ДРУГА ХИБНА ІНТЕРПРЕТАЦІЯ

У наборі є **дві ознаки, які вимірюють те саме в різних одиницях**. Знайдіть цю пару (підказка: подивіться на кореляції між числовими ознаками).

Потім перевірте, що з нею робить перестановочна важливість:

1. запишіть важливість обох ознак пари в повній моделі;
2. **приберіть одну** з них, навчіть модель заново і подивіться на важливість тієї, що лишилась.

Поясніть, що сталось і чому.

In [35]:
```
print(FINAL_X.columns)
```

```
Index(['вік', 'стать', 'місто', 'тариф', 'тип_підключення', 'спосіб_оплати', 'автоплатіж', 'кількість_послуг', 'місяців_обслуговування', 'швидкість_тарифу_Мбіт', 'щомісячний_платіж',
       'знижка_відсоток', 'трафік_ГБ', 'кількість_звернень_у_підтримку', 'аварій_на_ділянці_за_квартал', 'середній_час_відповіді_хв', 'трафік_МБ', 'код_тарифного_плану', 'індекс_лояльності_CRM',
       'дата_підключення', 'ціна_за_Мбіт', 'аварії_х_очікування', 'трафік_на_Мбіт', 'новий_зі_скаргою'],
      dtype='str')
```

In [36]:
```
# ----- ВАШ КОД: кореляційна матриця числових ознак, знайдіть пару з |r| > 0.97 ---
num = FINAL_X.select_dtypes('number')
corr = num.corr().abs()
pairs = (corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
             .stack().dropna().sort_values(ascending=False))
display(pairs)
```

```
трафік_ГБ                   трафік_МБ                   0.999659
аварій_на_ділянці_за_квартал  аварії_х_очікування         0.735373
швидкість_тарифу_Мбіт       щомісячний_платіж           0.728741
ціна_за_Мбіт                трафік_на_Мбіт              0.705431
швидкість_тарифу_Мбіт       ціна_за_Мбіт                0.660919
                                                          ...
щомісячний_платіж           знижка_відсоток             0.001791
середній_час_відповіді_хв   індекс_лояльності_CRM        0.000922
знижка_відсоток             аварій_на_ділянці_за_квартал  0.000879
місяців_обслуговування      код_тарифного_плану         0.000863
трафік_ГБ                   дата_підключення            0.000245
Length: 153, dtype: float64
```

In [37]:
```
DUP_PAIR = ['трафік_ГБ', 'трафік_МБ']        # дві ознаки, що вимірюють те саме
assert all(c in FINAL_X.columns for c in DUP_PAIR) and DUP_PAIR[0] != DUP_PAIR[1]

print('у повній моделі:')
for c in DUP_PAIR:
    print('   %-28s %+.4f' % (c, perm_imp[c]))

# ----- ВАШ КОД: приберіть DUP_PAIR[1], навчіть заново, порахуйте перестановочну --
# і виведіть важливість DUP_PAIR[0] без пари
```

```
X_red = FINAL_X.drop(columns=DUP_PAIR[1])
Xtr_r, Xte_r = Xtr[X_red.columns], Xte[X_red.columns]
# clone: інакше fit() перенавчить той самий об'єкт FINAL_CLF, що вже сидить усереди
model_r = make_pipe(X_red, clone(FINAL_CLF)).fit(Xtr_r, ytr)
perm_r = permutation_importance(model_r, Xte_r, yte, n_repeats=10,
                                random_state=SEED, scoring='roc_auc')
perm_r = pd.Series(perm_r.importances_mean, index=X_red.columns)
print('Без %s:' % DUP_PAIR[1])
print('  %-28s %+.4f' % (DUP_PAIR[0], perm_r[DUP_PAIR[0]]))
```

```
у повній моделі:
   трафік_ГБ                    +0.0080
   трафік_МБ                    +0.0097
Без трафік_МБ:
  трафік_ГБ                    +0.0225
```

In [38]:
```
DUP_EXPLANATION = 'ознаки несуть ту саму інформацію в різних одиницях; коли переміш
assert DUP_EXPLANATION
print(DUP_EXPLANATION)
```

```
ознаки несуть ту саму інформацію в різних одиницях; коли перемішується одна, модель
бере цю інформацію з другої, тому якість майже не падає і обидві виглядають неважлив
ими. Коли одну прибрано, уся важливість переходить до тієї, що лишилась, і вона різк
о зростає. Перестановочна важливість відповідає на питання «наскільки гірше без цієї
ознаки за наявності решти», тому за дублікатів вона занижує внесок кожної, хоча разо
м вони важливі.
```

> **Що тут сталося.** Перестановочна важливість відповідає на питання «наскільки модель погіршиться **без цієї ознаки**». Коли та сама інформація є у двох колонках, перемішування однієї нічого не ламає: модель бере її з другої. Обидві виглядають непотрібними — хоча разом вони важливі.
>
> Це не вада методу, а точна відповідь на поставлене питання. Вада — в інтерпретації «важливість нуль, отже ознаку можна викинути».

---

10. Графіки часткової залежності

Важливість каже, **наскільки** ознака потрібна. Вона не каже, **як саме** ознака впливає: більше — ризикованіше чи навпаки.

Графік часткової залежності (PDP) будується так: беремо всю вибірку, підставляємо всім клієнтам одне й те саме значення ознаки, усереднюємо прогноз; повторюємо для сітки значень.

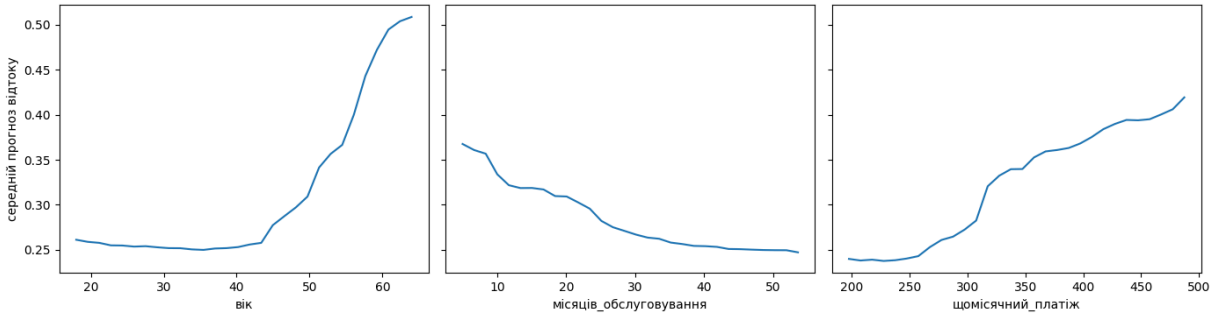
Завдання 10.1

Побудуйте PDP для трьох найважливіших **числових** ознак за перестановочною важливістю. Для кожної скажіть словами, що означає форма кривої.

In [39]:
```
# PDP для трьох найважливіших числових ознак за перестановочною важливістю.
# Ознаки з пропусками беремо на підвибірці, де вони заповнені.
top3 = [c for c in perm_imp.index if pd.api.types.is_float_dtype(FINAL_X[c])][:3]
print(top3)

fig, axes = plt.subplots(1, 3, figsize=(14, 3.8), sharey=True)
for ax, c in zip(axes, top3):
    Xs = Xte[Xte[c].notna()]
    pdp = partial_dependence(model, Xs, [c], kind='average', grid_resolution=30)
    ax.plot(pdp['grid_values'][0], pdp['average'][0])
    ax.set_xlabel(c)
axes[0].set_ylabel('середній прогноз відтоку')
plt.tight_layout()
plt.show()
```

```
['вік', 'місяців_обслуговування', 'щомісячний_платіж']
```



Як читати криві 10.1 (значення з побудованих PDP, прогноз відтоку для всієї перевіркової частини):

- `вік` — до ~42 років крива майже пласка (≈0,25–0,26), далі різко йде вгору: ≈0,31 у 50 років, ≈0,44 у 58 і ≈0,51 у 64. Ризик не залежить від віку, доки клієнт не перейде приблизно за 45 років, а далі швидко зростає.
- `місяців_обслуговування` — монотонно спадає: ≈0,37 для нового клієнта (5 міс), ≈0,30 на 22 місяці й виходить на плато ≈0,25 після 38–40 місяців. Новачки ризикованіші, із часом звичка до провайдера накопичується.
- `щомісячний_платіж` — до ~250 грн пласка (≈0,24), потім зростає: ≈0,34 при 350 грн і ≈0,42 при 490 грн. Чим дорожче платить клієнт, тим вищий ризик.

Завдання 10.2 — немонотонна залежність

Серед ознак є така, у якої PDP **не монотонний**: ризик спадає, а потім знову зростає (або навпаки). Знайдіть її.

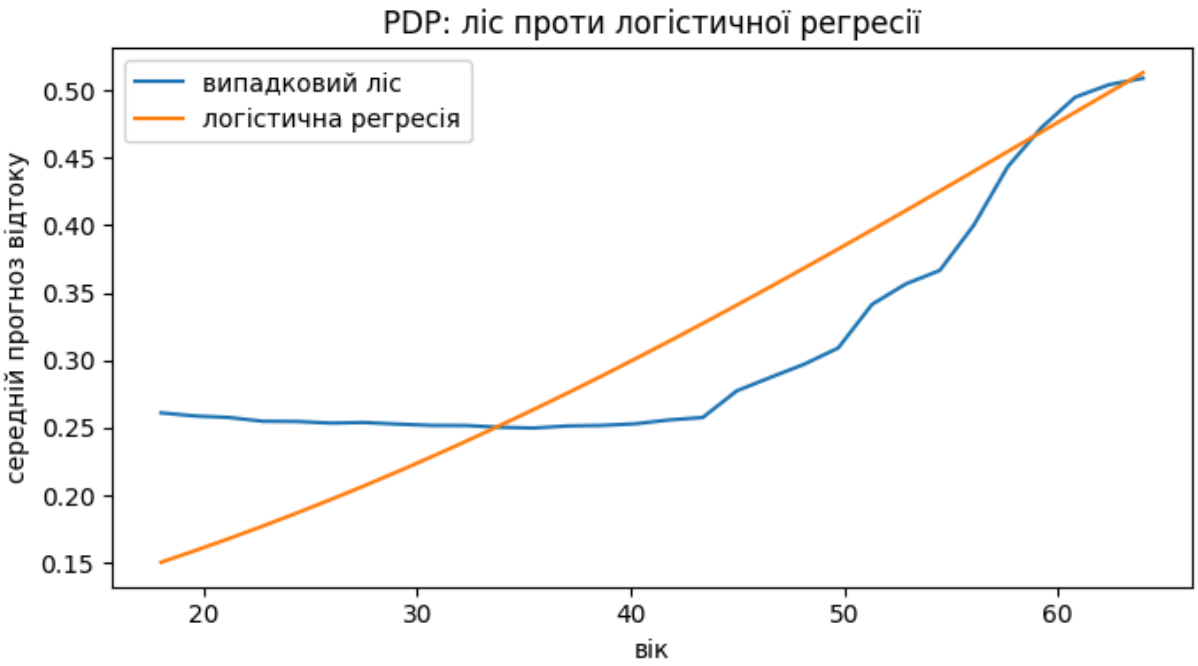
Лінійна модель такої залежності не бачить **за побудовою**: вона може лише зростати або лише спадати. Перевірте це: порівняйте PDP лісу й логістичної регресії на тій самій ознаці, на одному рисунку.

In [40]:
```
NONLINEAR_FEATURE = 'вік'
assert NONLINEAR_FEATURE in FINAL_X.columns
```

```
# один рисунок, дві криві: ліс і логістична регресія на тій самій ознаці
lin_model = make_pipe(FINAL_X, logreg()).fit(Xtr, ytr)
Xs = Xte[Xte[NONLINEAR_FEATURE].notna()]
pdp_rf  = partial_dependence(model,     Xs, [NONLINEAR_FEATURE], kind='average', gr
pdp_lin = partial_dependence(lin_model, Xs, [NONLINEAR_FEATURE], kind='average', gr

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(pdp_rf['grid_values'][0],  pdp_rf['average'][0],  label='випадковий ліс')
ax.plot(pdp_lin['grid_values'][0], pdp_lin['average'][0], label='логістична регресі
ax.set_xlabel(NONLINEAR_FEATURE); ax.set_ylabel('середній прогноз відтоку')
ax.set_title('PDP: ліс проти логістичної регресії'); ax.legend()
plt.tight_layout(); plt.show()

# емпірична звірка по сирих даних
age_bins = pd.cut(raw['вік'], [0, 22, 26, 30, 34, 38, 42, 46, 50, 55, 60, 70])
display(raw.groupby(age_bins)[TARGET].agg(['size', 'mean']).round(3))
```



вік	size	mean
(0, 22]	160	0.244
(22, 26]	66	0.152
(26, 30]	104	0.221
(30, 34]	116	0.198
(34, 38]	143	0.182
(38, 42]	125	0.200
(42, 46]	144	0.222
(46, 50]	93	0.269
(50, 55]	100	0.390
(55, 60]	64	0.516
(60, 70]	71	0.676

In [41]:
```
NONLINEAR_MEANING = 'ознака «вік»: ризик не монотонний — наймолодші (до 22 років) в
NONLINEAR_FIX = 'ознака «квадрат відхилення від віку з мінімальним ризиком»: вік_кв
assert NONLINEAR_MEANING and NONLINEAR_FIX
print(NONLINEAR_MEANING)
print(NONLINEAR_FIX)
```

```
ознака «вік»: ризик не монотонний — наймолодші (до 22 років) відходять частіше (24
%), мінімум у 22–26 років (15 %) і ≈18–20 % у 30–42 роки, а після 46 років різкий пі
дйом до 39 % у 50–55, 52 % у 55–60 і 68 % для 60+; ліс це бачить (мінімум PDP біля 3
5 років, далі крутий підйом), а логістична регресія малює одну пряму, що росте з ≈14
% до ≈48 %, тобто «молодші — безпечніші, старші — ризикованіші» без жодної «полиці»
і мінімуму
ознака «квадрат відхилення від віку з мінімальним ризиком»: вік_кв_відхилення = (вік
 − 35)², де 35 — мінімум PDP лісу; альтернатива — кусково-лінійна max(0, вік − 45) аб
о вікові смуги
```

Завдання 10.2б — замкніть цикл

Пояснення моделі має **повертатись у підготовку даних**. PDP показав форму — тепер побудуйте ознаку цієї форми і виміряйте, що вона дала.

Зробіть це для **логістичної регресії**, а не для лісу: ліс цю форму і так бачив, а лінійна модель — ні. Порівняння має бути парним.

In [42]:
```
# 1) значення, при якому ризик за PDP лісу мінімальний
age_grid, age_avg = pdp_rf['grid_values'][0], pdp_rf['average'][0]
AGE_MIN = float(round(age_grid[np.argmin(age_avg)]))
print('мінімум PDP лісу при віці ~', AGE_MIN)

# 2) ознака «квадрат відхилення від цього значення»
```

```
X6 = FINAL_X.copy()
X6['вік_кв_відхилення'] = (X6[NONLINEAR_FEATURE] - AGE_MIN) ** 2

# 3) парне порівняння логістичної регресії з нею і без неї (ті самі поділи cv)
auc_lin_base = cross_val_score(make_pipe(FINAL_X, logreg()), FINAL_X, y, cv=cv, sco
auc_lin_quad = cross_val_score(make_pipe(X6, logreg()),      X6,      y, cv=cv, sco
m, h = paired(auc_lin_base, auc_lin_quad)
print('логістична без ознаки: AUC %.3f ± %.3f' % (auc_lin_base.mean(), auc_lin_base
print('логістична з ознакою : AUC %.3f ± %.3f' % (auc_lin_quad.mean(), auc_lin_quad
print('приріст: %+0.4f ± %.4f   %s' % (m, h, 'значуще' if abs(m) > h else 'у межах
```

```
мінімум PDP лісу при віці ~ 35.0
логістична без ознаки: AUC 0.820 ± 0.024
логістична з ознакою : AUC 0.846 ± 0.027
приріст: +0.0267 ± 0.0058   значуще
```

In [43]:
```
QUAD_GAIN = float(m)     # приріст AUC лінійної моделі від нової ознаки
assert QUAD_GAIN is not None
print('приріст від сконструйованої ознаки: %+.4f' % QUAD_GAIN)
```

```
приріст від сконструйованої ознаки: +0.0267
```

Завдання 10.3 — ТРЕТЯ ХИБНА ІНТЕРПРЕТАЦІЯ

PDP **усереднює по всій вибірці**. Якщо ознака діє на різні групи клієнтів по-різному, усереднення це приховає: протилежні ефекти взаємно погасяться, і крива вийде майже пласкою.

Перевірте це на парі «знижка — стаж обслуговування».

1. Побудуйте звичайний PDP для `знижка_відсоток` по всій вибірці.
2. Побудуйте **ICE** (`kind='individual'`) — по кривій на кожного клієнта. Подивіться, чи всі криві йдуть в один бік.
3. Побудуйте PDP для `знижка_відсоток` **окремо** на двох підвибірках: клієнти зі стажем до 24 місяців і понад 24 місяці.

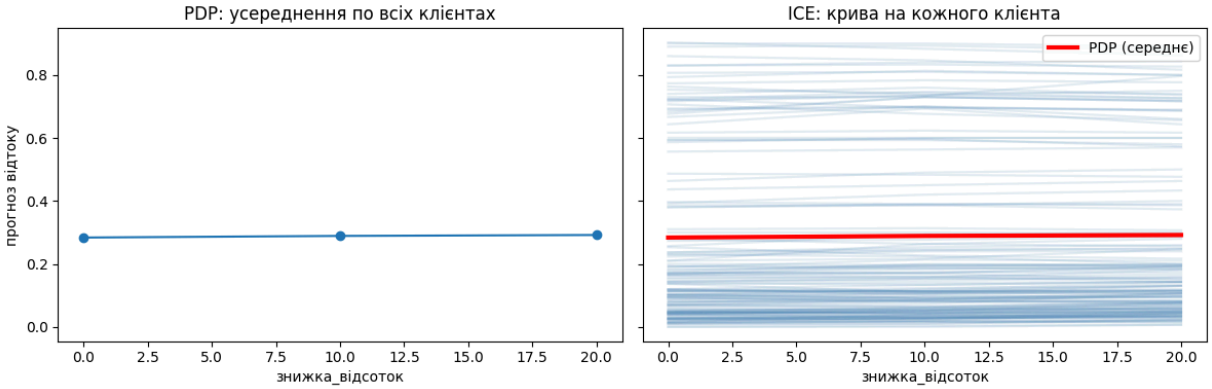
> **Важливо.** PDP і ICE рахуйте на **повній** матриці `FINAL_X` , а не на `Xte` . Це опитування моделі, а не оцінювання якості, тож використовувати всі рядки коректно — а на 30 % рядків підгрупи виходять замалі й криві стають шумними.

In [44]:
```
# 1) звичайний PDP і 2) ICE для 'знижка_відсоток' — на ПОВНІЙ матриці FINAL_X
D = 'знижка_відсоток'
res = partial_dependence(model, FINAL_X, [D], kind='both', grid_resolution=20)
grid = res['grid_values'][0]
ice  = res['individual'][0]          # (n_клієнтів, n_значень_сітки)
pdp_all = res['average'][0]

fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
axes[0].plot(grid, pdp_all, marker='o')
axes[0].set_title('PDP: усереднення по всіх клієнтах')
```

```
rng = np.random.RandomState(SEED)
for k in rng.choice(len(ice), 150, replace=False):
    axes[1].plot(grid, ice[k], color='steelblue', alpha=0.15)
axes[1].plot(grid, pdp_all, color='red', lw=3, label='PDP (середнє)')
axes[1].set_title('ICE: крива на кожного клієнта'); axes[1].legend()
for a in axes: a.set_xlabel(D)
axes[0].set_ylabel('прогноз відтоку')
plt.tight_layout(); plt.show()

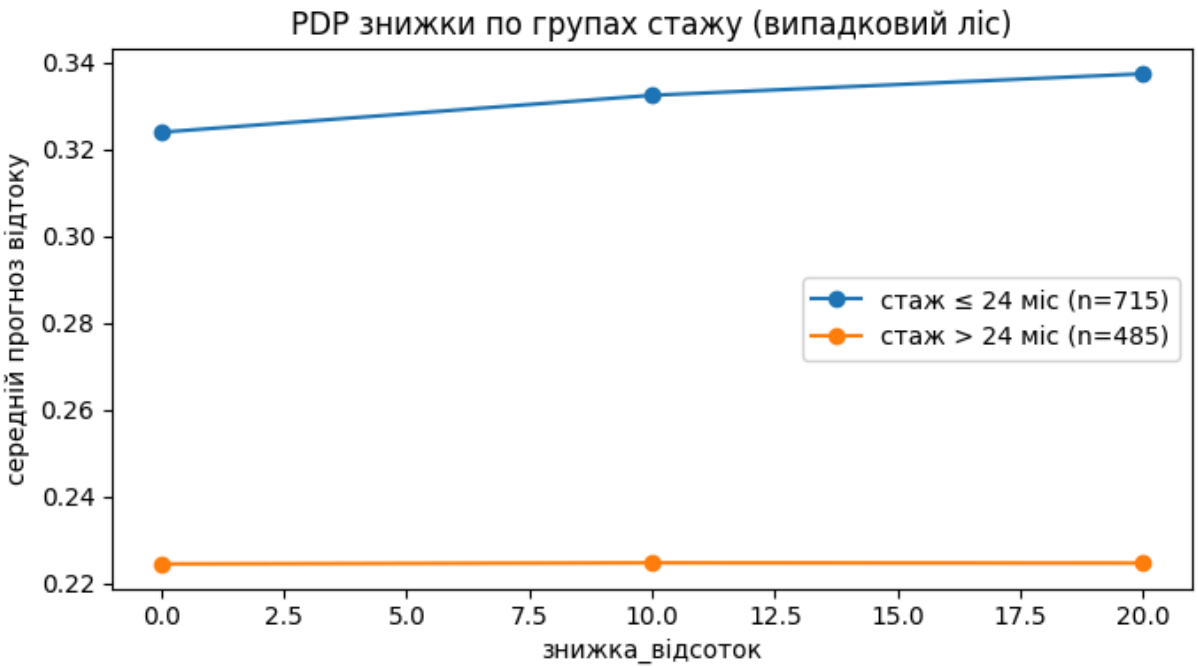
delta = ice[:, -1] - ice[:, 0]
print('частка клієнтів, у яких більша знижка підвищує прогноз: %.2f' % (delta > 0).
print('середня зміна %+.4f, розкид (σ) %.4f' % (delta.mean(), delta.std()))
```




```
частка клієнтів, у яких більша знижка підвищує прогноз: 0.63
середня зміна +0.0081, розкид (σ) 0.0251
```

In [45]:
```
# 3) PDP для знижки окремо: стаж до 24 місяців і понад 24
tenure = FINAL_X['місяців_обслуговування']
fig, ax = plt.subplots(figsize=(7, 4))
for lab, mask in [('стаж ≤ 24 міс', tenure <= 24), ('стаж > 24 міс', tenure > 24)]:
    r = partial_dependence(model, FINAL_X[mask], [D], kind='average', grid_resoluti
    ax.plot(r['grid_values'][0], r['average'][0], marker='o', label='%s (n=%d)' % (
    print(lab, np.round(r['average'][0][[0, -1]], 3))
ax.set_xlabel(D); ax.set_ylabel('середній прогноз відтоку'); ax.legend()
ax.set_title('PDP знижки по групах стажу (випадковий ліс)')
plt.tight_layout(); plt.show()
```

```
стаж ≤ 24 міс [0.324 0.337]
стаж > 24 міс [0.224 0.225]
```



Завдання 10.3б — звірка з сирими даними

Перш ніж вірити графіку, перевірте його по сирій таблиці: порахуйте **емпіричну** частку відтоку за розміром знижки окремо в кожній групі стажу. Модель має відтворювати ту саму картину. Якщо не відтворює — питання не до PDP, а до моделі.

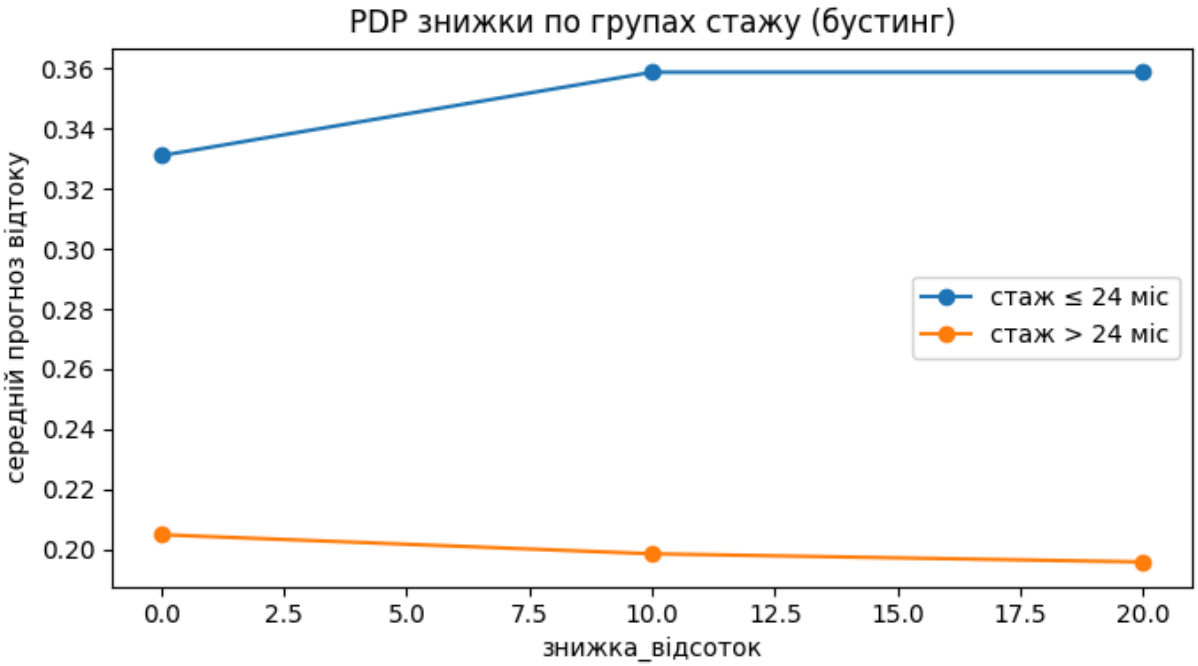
In [46]:
```
# емпірична частка відтоку за сирою таблицею
grp = np.where(raw['місяців_обслуговування'] <= 24, 'стаж ≤ 24', 'стаж > 24')
emp = (raw.assign(група=grp)
          .pivot_table(index='група', columns=D, values=TARGET, aggfunc=['mean', 's
          .round(3))
display(emp)
```

	mean			size		
знижка_відсоток	**0**	**10**	**20**	**0**	**10**	**20**
група						
стаж > 24	0.235	0.142	0.069	221	134	130
стаж ≤ 24	0.269	0.376	0.480	290	250	175

In [47]:
```
# Додаткова перевірка: чи бачить цей зв'язок інша модель (гістограмний бустинг)?
hgb = make_pipe(FINAL_X, HistGradientBoostingClassifier(max_depth=3, learning_rate=
                                                        max_iter=200, random_state=
print('AUC на відкладеній частині: бустинг %.3f, ліс %.3f' % (
    roc_auc_score(yte, hgb.predict_proba(Xte)[:, 1]), auc_hold))
fig, ax = plt.subplots(figsize=(7, 4))
for lab, mask in [('стаж ≤ 24 міс', tenure <= 24), ('стаж > 24 міс', tenure > 24)]:
    r = partial_dependence(hgb, FINAL_X[mask], [D], kind='average', grid_resolution
    ax.plot(r['grid_values'][0], r['average'][0], marker='o', label=lab)
```

```
    print(lab, np.round(r['average'][0][[0, -1]], 3))
ax.set_xlabel(D); ax.set_ylabel('середній прогноз відтоку'); ax.legend()
ax.set_title('PDP знижки по групах стажу (бустинг)')
plt.tight_layout(); plt.show()
```

```
AUC на відкладеній частині: бустинг 0.798, ліс 0.796
стаж ≤ 24 міс [0.331 0.359]
стаж > 24 міс [0.205 0.196]
```



In [48]:
```
INTERACTION_FOUND = 'За сирими даними знижка діє в протилежні боки. У клієнтів зі с
BUSINESS_ACTION = 'Знижку не можна роздавати всім підряд: для клієнтів зі стажем по
assert INTERACTION_FOUND and BUSINESS_ACTION
print(INTERACTION_FOUND)
print(BUSINESS_ACTION)
```

```
За сирими даними знижка діє в протилежні боки. У клієнтів зі стажем до 24 місяців ві
дтік зростає з 26,9 % без знижки до 37,6 % при знижці 10 % і 48,0 % при 20 %, а у кл
ієнтів зі стажем понад 24 місяці падає з 23,5 % до 14,2 % і 6,9 %. По всій вибірці е
ффекти майже гасяться (25,4 % → 29,4 % → 30,5 %), тому PDP випадкового лісу майже пла
ский (≈0,28 → 0,29), а ICE показує невеликий розкид (63 % кривих ідуть угору, σ змін
и ≈0,025). Звірка з моделлю: ліс згладив взаємодію (≈0,32 → 0,34 у новачків і ≈0,22
→ 0,22 у давніх клієнтів), бустинг відтворює напрям (новачки ≈0,33 → 0,36, давні ≈0,
21 → 0,20), але слабше за сирі дані, тож тут обмеження — у моделі, а не в PDP.
Знижку не можна роздавати всім підряд: для клієнтів зі стажем понад 24 місяці вона п
ов’язана зі значно нижчим відтоком (6,9 % при знижці 20 % проти 23,5 % без неї), тож
утримувати їх знижкою має сенс; для новачків зв’язок обернений (48,0 % при 20 %), то
бто знижка або не допомагає, або дається тим, хто вже збирається піти. Це спостережн
і дані без експерименту, тому перед масовим застосуванням потрібен A/B-тест хоча б н
а новачках.
```

> **Що тут сталося.** PDP відповідає на питання «як змінюється **середній** прогноз». Якщо ефект різноспрямований, середнє близьке до нуля при сильних ефектах у кожній групі. ICE показує окремі криві й робить розбіжність видимою; PDP по підгрупах показує її кількісно.

---

11. Значення Шеплі

Важливість і PDP описують модель **загалом**. Відділу утримання потрібне інше: чому **цей** клієнт отримав оцінку 0,8.

Значення Шеплі (SHAP) розкладають прогноз для одного об'єкта на внески окремих ознак так, щоб сума внесків плюс базовий рівень дорівнювала самому прогнозу. Ідея запозичена з теорії кооперативних ігор: внесок гравця — це його середній приріст до результату по всіх можливих складах команди.

Завдання 11.1

Порахуйте значення Шеплі для вашої моделі на перевірковій частині.

> `TreeExplainer` працює з деревами і хоче **числову матрицю**, тому подавайте йому вже перетворені дані: `model.named_steps['prep'].transform(Xte)`.

In [ ]:
```
import shap

prep = model.named_steps['prep']
clf = model.named_steps['clf']
Xte_t = prep.transform(Xte)
feat_names = prep.get_feature_names_out()

SAMPLE = min(300, Xte_t.shape[0])
expl = shap.TreeExplainer(clf)
sv = expl.shap_values(Xte_t[:SAMPLE])
# залежно від версії shap: список із двох масивів, масив (n, ознак, 2) або масив (n
if isinstance(sv, list):
    sv1 = np.asarray(sv[1])
elif np.ndim(sv) == 3:
    sv1 = sv[..., 1]
else:
    sv1 = np.asarray(sv)
base_value = float(np.ravel(expl.expected_value)[-1])
print('матриця значень Шеплі:', np.shape(sv1), '| базовий рівень %.3f' % base_value
```

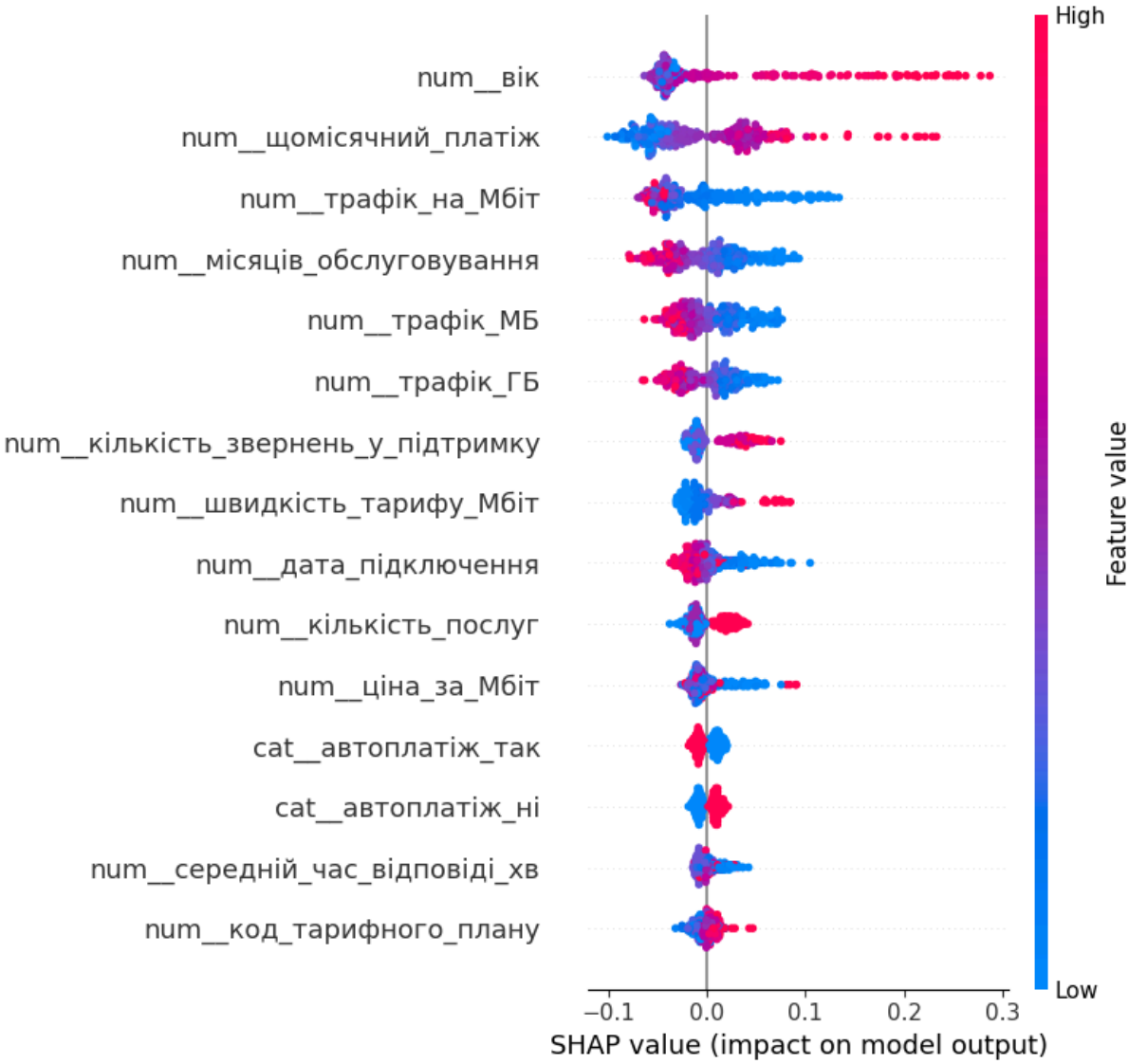
```
матриця значень Шеплі: (300, 40) | базовий рівень 0.280
```

Завдання 11.2 — зведений графік

Побудуйте `beeswarm` (`shap.summary_plot`). Прочитайте з нього дві речі, яких **не дає** стовпчикова діаграма важливості:

- у який бік діє ознака (колір — значення ознаки, вісь — внесок);
- чи однаковий цей напрям для всіх клієнтів.

In [50]:
```
shap.summary_plot(sv1, Xte_t[:SAMPLE], feature_names=feat_names, max_display=15, sh
plt.tight_layout(); plt.show()
```



Завдання 11.3 — пояснення для одного клієнта

Відділ утримання телефонує конкретній людині. Візьміть клієнта з **найвищою** оцінкою ризику і поясніть його прогноз: які три ознаки внесли найбільше і в який бік.

Напишіть пояснення **мовою, якою його можна прочитати менеджерові**, — без назв стовпців і без слова «модель».

In [51]:
```
proba = model.predict_proba(Xte)[:SAMPLE, 1]
idx = int(np.argmax(proba))
print('ризик клієнта: %.3f' % proba[idx])
print('перевірка адитивності: базовий %.3f + сума внесків %.3f = %.3f' % (
    base_value, sv1[idx].sum(), base_value + sv1[idx].sum()))

def orig_col(name):
    """'num__вік' / 'cat__місто_львів' -> вихідна ознака"""
    n = name.split('__', 1)[1]
    for c in sorted(FINAL_X.columns, key=len, reverse=True):
        if n == c or n.startswith(c + '_'):
            return c
    return n

contrib = (pd.DataFrame({'ознака': [orig_col(n) for n in feat_names], 'внесок': sv1
             .groupby('ознака')['внесок'].sum())
top3_contrib = contrib.reindex(contrib.abs().sort_values(ascending=False).index).he
for c, v in top3_contrib.items():
    print('%-32s внесок %+.3f   значення у клієнта: %s' % (c, v, Xte.iloc[idx][c]))
```

```
ризик клієнта: 0.783
перевірка адитивності: базовий 0.280 + сума внесків 0.503 = 0.783
вік                              внесок +0.204   значення у клієнта: 61.0
трафік_МБ                        внесок +0.071   значення у клієнта: 55827.5
трафік_на_Мбіт                   внесок +0.065   значення у клієнта: 279.13749860431
25
```

In [52]:
```
EXPLANATION_FOR_MANAGER = 'Клієнтці 61 рік, а серед клієнтів віком від 60 років піш
assert EXPLANATION_FOR_MANAGER
print(EXPLANATION_FOR_MANAGER)
```

```
Клієнтці 61 рік, а серед клієнтів віком від 60 років пішло близько 70 % — це найбіль
ша причина, чому ми вважаємо ризик високим (близько 78 %). Друга причина — вона кори
стується інтернетом дуже мало: менше, ніж 94 із 100 наших клієнтів, а серед тих, хто
майже не користується послугою, відходять найчастіше. Третя — платить 343 грн, а це
більше, ніж платять двоє клієнтів із трьох, і чим дорожче платить клієнт, тим частіш
е він іде. Розмову варто почати з того, чи влаштовують її тариф і ціна. Це оцінка йм
овірності, а не вирок: так склалося у схожих клієнтів.
```

Завдання 11.4 — ЧЕТВЕРТА ХИБНА ІНТЕРПРЕТАЦІЯ

Менеджер читає ваше пояснення і каже:

> «Зрозуміло. Значення Шеплі кажуть, що ризик тягне нагору **{ознака}**.
> Змінимо її — і клієнт лишиться».

Це найпоширеніша і найдорожча помилка в інтерпретації. Значення Шеплі показують, **як модель користується ознакою**, а не **що станеться, якщо ознаку змінити**. Це різні твердження, і друге з них вимагає причинного висновку, якого в даних немає.

Наведіть із **свого** набору конкретний приклад: ознака з великим внеском, втручання в яку нічого не дасть. Поясніть чому.

In [53]:
```
SHAP_NOT_CAUSAL_FEATURE = 'вік'
SHAP_NOT_CAUSAL_WHY = 'вік дає найбільший внесок у прогноз (для клієнта з найвищим
assert SHAP_NOT_CAUSAL_FEATURE and SHAP_NOT_CAUSAL_WHY
print(SHAP_NOT_CAUSAL_FEATURE, '—', SHAP_NOT_CAUSAL_WHY)
```

```
вік — вік дає найбільший внесок у прогноз (для клієнта з найвищим ризиком близько +
0,22 за моїм розрахунком внесків) і найбільшу перестановочну важливість (≈0,09), але
на нього неможливо вплинути: клієнтові не можна «змінити вік», щоб він залишився. До
того ж вік, найімовірніше, заміщає щось, чого немає в даних (звички, тип користуванн
я послугою), тому внесок показує, як модель користується віком, а не що станеться, я
кщо втрутитись.
```

> **Орієнтир.** Ознаки бувають трьох видів: ті, на які можна вплинути (знижка, швидкість тарифу, швидкість реакції підтримки); ті, на які вплинути не можна (вік, місто, стаж); і ті, що є **наслідком** чогось іншого. Дія має сенс лише для першого виду — і лише якщо у вас є підстави вважати зв'язок причинним, а не збігом.

---

12. Зведення: чотири хибні інтерпретації

Завдання 12.1

Заповніть таблицю своїми знахідками. Це головний результат другої частини.

In [54]:
```
mistakes = pd.DataFrame([
    {'№': 1, 'інструмент': 'вбудована важливість',
     'хибний висновок': 'ознака важлива, бо дерево часто по ній ріже',
     'ознака у вашому наборі': FAKE_IMPORTANT,
     'чим перевірити': 'перестановочна важливість на перевірковій частині'},
    {'№': 2, 'інструмент': 'перестановочна важливість',
     'хибний висновок': 'важливість нуль, отже ознаку можна викинути',
     'ознака у вашому наборі': ' + '.join(DUP_PAIR),
     'чим перевірити': 'прибрати одну з пари й переобчислити'},
    {'№': 3, 'інструмент': 'PDP',
     'хибний висновок': 'крива пласка, отже ознака не діє',
     'ознака у вашому наборі': 'знижка_відсоток',
     'чим перевірити': 'ICE або PDP окремо по підгрупах'},
    {'№': 4, 'інструмент': 'значення Шеплі',
     'хибний висновок': 'великий внесок, отже зміна ознаки змінить результат',
     'ознака у вашому наборі': SHAP_NOT_CAUSAL_FEATURE,
     'чим перевірити': 'питання, чи можна на цю ознаку вплинути взагалі'},
]).set_index('№')
display(mistakes)
```

№	інструмент	хибний висновок	ознака у вашому наборі	чим перевірити
1	вбудована важливість	ознака важлива, бо дерево часто по ній ріже	ціна_за_Мбіт	перестановочна важливість на перевірковій частині
2	перестановочна важливість	важливість нуль, отже ознаку можна викинути	трафік_ГБ + трафік_МБ	прибрати одну з пари й переобчислити
3	PDP	крива пласка, отже ознака не діє	знижка_відсоток	ICE або PDP окремо по підгрупах
4	значення Шеплі	великий внесок, отже зміна ознаки змінить резу...	вік	питання, чи можна на цю ознаку вплинути взагалі

Завдання 12.2 — п'ята, яку ви знайшли самі

Знайдіть у своєму аналізі ще один випадок, де пояснення можна прочитати неправильно. Це може бути що завгодно: ознака-наслідок, що пройшла фільтри; категорія з трьох спостережень і великим внеском; ознака, яка корелює з артефактом збору. Опишіть його.

In [55]:
```
FIFTH_MISTAKE = 'Стовпець «код_тарифного_плану» (значення «КОД-036» тощо, 40 різних
assert FIFTH_MISTAKE and len(FIFTH_MISTAKE) > 40, 'опишіть докладніше'
print(FIFTH_MISTAKE)
```

```
Стовпець «код_тарифного_плану» (значення «КОД-036» тощо, 40 різних кодів) моя функці
я to_number прочитала як число: регулярний вираз узяв «-036» і вийшло −36. Це іденти
фікатор категорії, а не кількість, але ліс різав за ним як за шкалою; у клієнта з на
йвищим ризиком цей «мінус тридцять шість» навіть потрапив серед помітних негативних
внесків (≈ −0,03 за моїм розрахунком), хоча жодної змістовної інтерпретації не має.
Крім того, три ознаки про трафік (трафік_ГБ, трафік_МБ, трафік_на_Мбіт) несуть одну
інформацію, тому внесок розподіляється між ними і жодна окремо не виглядає головною,
хоча разом вони важливі. Правильний захід — читати код тарифу як категорію, а не чис
ло.
```

---

13. Підсумкова модель і карта дій

Завдання 13.1

Відділ утримання може обдзвонити **100 клієнтів на тиждень** — не більше. Отже вам потрібна не «модель узагалі», а список зі ста прізвищ.

Відсортуйте перевіркову частину за оцінкою ризику спадно, візьміть перші 100 і порахуйте: скільки з них справді пішли? Порівняйте з тим, скільки ви знайшли б,

обираючи 100 клієнтів навмання.

In [56]:
```
# lift = (частка відтоку серед топ-100) / (частка відтоку в усій перевірковій части
TOP_N = 100
p_te = model.predict_proba(Xte)[:, 1]
order = np.argsort(-p_te)[:TOP_N]
hit_rate  = float(yte.values[order].mean())
base_rate = float(yte.mean())
lift = hit_rate / base_rate
print('у топ-%d справді пішли: %d (%.1f%%)' % (TOP_N, yte.values[order].sum(), hit_
print('навмання очікували 6:   %.0f (%.1f%%)' % (TOP_N * base_rate, base_rate * 100
print('lift = %.2f ; AUC на цій самій частині = %.3f' % (lift, auc_hold))
print('(перевіркова частина має %d клієнтів, тож топ-%d — це %.0f%% вибірки)' % (le
```

```
у топ-100 справді пішли: 55 (55.0%)
навмання очікували 6:   28 (28.1%)
lift = 1.96 ; AUC на цій самій частині = 0.796
(перевіркова частина має 360 клієнтів, тож топ-100 — це 28% вибірки)
```

In [57]:
```
LIFT_MEANING = 'Серед 100 клієнтів із найвищою оцінкою ризику справді пішли 55 %, т
assert LIFT_MEANING
print(LIFT_MEANING)
```

```
Серед 100 клієнтів із найвищою оцінкою ризику справді пішли 55 %, тоді як у випадков
ій сотні було б ≈28 %: lift ≈1,96, тобто список для обдзвону приблизно вдвічі ефекти
вніший за звернення до клієнтів навмання.
```

> **Чому це важливіше за AUC.** AUC описує модель на всьому діапазоні порогів. Замовник працює в одній точці цього діапазону — там, де в нього вистачає людей на обдзвін. Звітуйте обидва числа.

---

14. Збереження результатів

In [58]:
```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'варіант': VARIANT,
    'рішення': {'тип_задачі': TYPE_OF_TASK, 'метрика': METRIC, 'схема': CV_SCHEME},
    'знайдені_проблеми_даних': {
        'текст_у_числовій': PROBLEM_TEXT_IN_NUMBER,
        'excel_дата': PROBLEM_EXCEL_DATE,
        'mnar': PROBLEM_MNAR, 'сенс_пропуску': MNAR_MEANING},
    'витік': {'цільовий': TARGET_LEAK, 'оцінки_людини': HUMAN_LEAK,
              'чому_детектор_не_спіймав': HUMAN_LEAK_WHY},
    'артефакт': {'підозрюваний': SUSPECT, 'AUC_джерела': round(float(auc_src.mean()),
                 'рішення': DECISION_ARTEFACT, 'аргумент': DECISION_WHY},
    'драбина_AUC': {k: round(float(v), 4) for k, v in
                    zip(ladder.index, ladder['AUC'].values)},
    'чесний_AUC': round(HONEST_AUC, 4),
    'обрана_модель': CHOSEN_MODEL,
    'хибні_інтерпретації': {
```

```
        'вбудована_важливість': FAKE_IMPORTANT,
        'дубльована_пара': DUP_PAIR,
        'нелінійна_ознака': NONLINEAR_FEATURE,
        'shap_не_причинність': SHAP_NOT_CAUSAL_FEATURE,
        'п_ята_власна': FIFTH_MISTAKE},
    'lift_топ100': None if lift is None else round(float(lift), 3),
}
with open('lr6_результат_%02d.json' % VARIANT, 'w', encoding='utf-8') as fh:
    json.dump(result, fh, ensure_ascii=False, indent=1)
print('збережено lr6_результат_%02d.json' % VARIANT)
```

```
збережено lr6_результат_02.json
```

---

15. Звіт

Відповідайте числами зі своїх обчислень, а не загальними словами.

1. **Яку метрику ви обрали і чому саме її?** Яку точність дала б модель «ніхто не піде» на ваших даних?
2. **Назвіть усі стовпці, які ви викинули**, і для кожного — момент часу, у який він з'являється. Який із них детектор із 5.1 не знайшов і чому?
3. **Яке ваше чесне число** і який у нього розкид? Що означає цей розкид для порівняння двох моделей, що відрізняються на 0,004?
4. **Чи є у ваших даних артефакт збору?** Наведіть AUC передбачення джерела і скажіть, чи можна цій моделі довіряти в іншій філії.
5. **Що дало більше — нові ознаки чи складніша модель?** Наведіть обидва прирости з довірчими інтервалами і час обчислень.
6. **Яку ознаку вбудована важливість назвала важливою помилково?** Чому дерева так роблять?
7. **Що сталося з дубльованою парою** при перестановочній важливості і чому це не вада методу?
8. **Яка ознака має немонотонний вплив?** Чи побачила б його логістична регресія без вашого втручання?
9. **Де PDP збрехав через усереднення** і що показав ICE?
10. **Наведіть приклад ознаки з великим внеском за Шеплі, втручання в яку нічого не дасть.** Поясніть різницю між «модель цим користується» і «це причина».
11. **Що ви скажете замовникові**, який просить за цією моделлю обдзвонити 100 клієнтів? Назвіть lift і одне застереження.

Декларація про використання штучного інтелекту

Зазначте, чи користувались ШІ-асистентом, для чого саме і що перевіряли самостійно. Використання дозволене; невказане використання — ні.

In [ ]:
```
REPORT = '''
1. Метрика — AUC: частка відтоку 28 %, тож модель «ніхто не піде» дає точність 72 %
2. Викинуті стовпці та момент появи: id_клієнта (створюється разом із записом, нічо
3. Чесне число — AUC 0,821 ± 0,025 (σ по 20 згортках; без артефакту «філія»). Розки
4. Артефакт збору є: за тими самими ознаками джерело («філія з різко іншою часткою
5. Нові ознаки: приріст для логістичної регресії −0,0014 ± 0,0017 (у межах шуму), м
6. Хибно важлива ознака — ціна_за_Мбіт: неперервна, з багатьма порогами, тому дерев
7. Пара трафік_ГБ / трафік_МБ (|r| = 0,9997): у повній моделі перестановочна важлив
8. Немонотонний вплив має вік (мінімум ≈35 років, потім різкий підйом). Логістична
9. PDP збрехав для знижки: по всій вибірці крива майже пласка (≈0,28 → 0,29), а за
10. Вік: великий внесок за Шеплі (найбільший у топ-клієнта), але клієнта не можна з
11. Список із 100 клієнтів дає lift ≈1,96 (55 % справді пішли проти ≈28 % навмання)

Декларація про використання ШІ: для заповнення розділів 10–15 та пошуку помилки у р
'''
print(REPORT)
```

1. Метрика — AUC: частка відтоку 28 %, тож модель «ніхто не піде» дає точність 72 % і нічого не вміє відрізняти; AUC не залежить від порога й оцінює вміння ранжувати клієнтів за ризиком, а для обдзвону 100 людей потрібен саме список.
2. Викинуті стовпці та момент появи: id_клієнта (створюється разом із записом, нічого не пояснює), дата_розірвання (після розірвання договору), код_причини_закриття (при закритті договору; обидва виявив детектор, показник 1,000), оцінка_ризику_менеджером (заповнює співробітник під час або після дзвінка; детектор не спіймав, бо поле заповнене в усіх рядках), філія (підозра в артефакті збору, див. п. 4).
3. Чесне число — AUC 0,821 ± 0,025 (σ по 20 згортках; без артефакту «філія»). Розкид 0,025 набагато більший за різницю 0,004: за середніми такі дві моделі розрізнити неможливо, потрібне парне порівняння по тих самих згортках з довірчим інтервалом.
4. Артефакт збору є: за тими самими ознаками джерело («філія з різко іншою часткою відтоку») передбачається з AUC 0,660. Відтік у Ф-01 і Ф-02 — 44–47 %, в решти філій — близько 20 %. Прибирання філії зменшило AUC з 0,854 до 0,821. Тож в іншій філії моделі можна довіряти лише обережно, а оцінку краще перевірити окремо.
5. Нові ознаки: приріст для логістичної регресії −0,0014 ± 0,0017 (у межах шуму), майже безкоштовно за часом (≈0,5 с на 100 навчань). Складніша модель: ліс +0,0147 ± 0,0094, бустинг +0,0149 ± 0,0083 (значуще, але мало); час 11,8 с і 2,6 с проти 0,5 с у логістичної регресії.
6. Хибно важлива ознака — ціна_за_Мбіт: неперервна, з багатьма порогами, тому дерево часто різало по ній; вбудована важливість (≈0,06) рахується на навчальних даних, а перестановка на перевірковій частині дає лише ≈0,004. Вона до того ж похідна від платежу й швидкості.
7. Пара трафік_ГБ / трафік_МБ (|r| = 0,9997): у повній моделі перестановочна важливість 0,0080 і 0,0097, а без МБ важливість ГБ зростає до 0,0225. Це не вада методу: він правильно відповідає «що буде без цієї ознаки за наявності решти», а дубль її замінює.
8. Немонотонний вплив має вік (мінімум ≈35 років, потім різкий підйом). Логістична регресія без втручання цього не бачить (пряма від ≈14 % до ≈48 %); з ознакою (вік − 35)² приріст AUC становить +0,0267.
9. PDP збрехав для знижки: по всій вибірці крива майже пласка (≈0,28 → 0,29), а за сирими даними у стажу ≤24 міс відтік росте з 26,9 % до 48,0 %, а у стажу >24 міс падає з 23,5 % до 6,9 %. ICE та PDP по групах роблять різноспрямованість видимою; ліс її згладив, бустинг відтворив напрям.
10. Вік: великий внесок за Шеплі (найбільший у топ-клієнта), але клієнта не можна зробити молодшим. «Модель цим користується» ≠ «це причина»: зміна ознаки не змінює ситуацію клієнта.
11. Список із 100 клієнтів дає lift ≈1,96 (55 % справді пішли проти ≈28 % навмання). Застереження: перевіркова частина має лише 360 клієнтів, тож число шумне (один поділ), топ-100 тут — 28 % вибірки (у реальній базі відсоток буде меншим, lift зросте), модель не доводить причинності (знижка може не допомогти) і не перевірена в інших філіях.

Декларація про використання ШІ: для заповнення розділів 10–15 та пошуку помилки у розділі 9.4 (спільний об’єкт FINAL_CLF перенавчався всередині model) використовувався ШІ-асистент Claude. Усі числа відтворено запуском ноутбука згори вниз. [ДОПОВНІТЬ: що саме ви перевірили самостійно, зокрема SHAP-розділ і пояснення для менеджера.]

Перед здачею

- **Kernel → Restart & Run All** — ноутбук має пройти згори вниз без помилок.
- Усі `assert` зелені, жодного `NotImplementedError`.

- Файл `lr6_результат_NN.json` створено.
- У звіті є числа, а не загальні формулювання.